Vorlesung
Informationsvisualisierung

Kapitel 9
SOFTWAREVISUALISIERUNG



Lecture
Information Visualization

Chapter 9
SOFTWARE VISUALIZATION

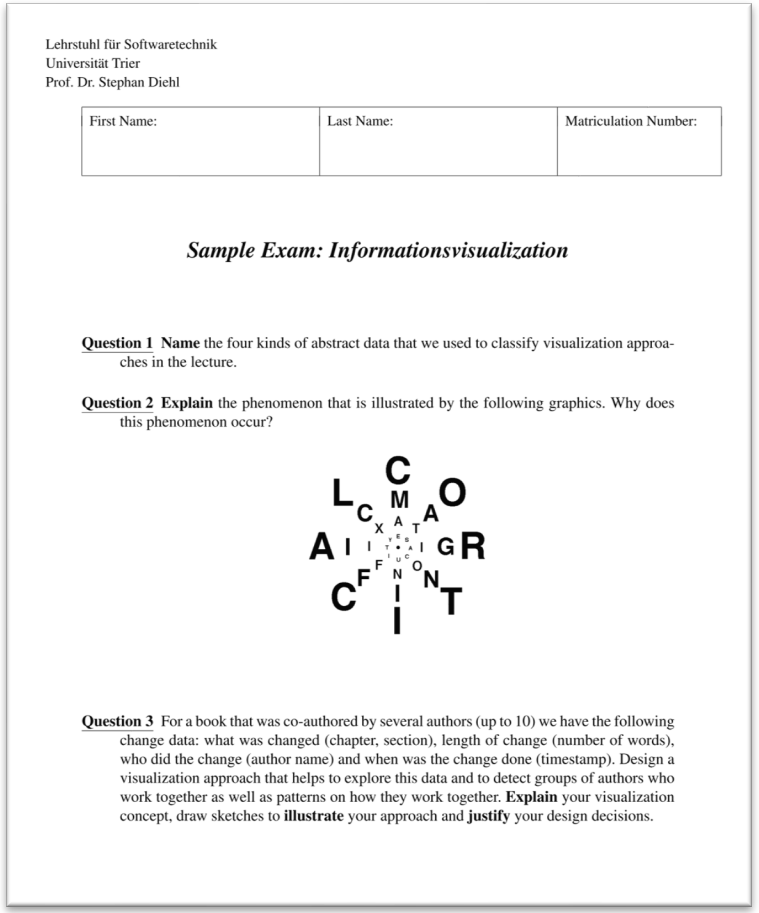
Prof. Dr. Stephan Diehl

Universität Trier – Sommersemester 2026

There will be a tutorial next Tuesday !!!



Sample exam is available in StudIP



Lehrstuhl für Softwaretechnik
Universität Trier
Prof. Dr. Stephan Diehl

First Name:	Last Name:	Matriculation Number:

Sample Exam: Informationsvisualization

Question 1 **Name** the four kinds of abstract data that we used to classify visualization approaches in the lecture.

Question 2 **Explain** the phenomenon that is illustrated by the following graphics. Why does this phenomenon occur?

Question 3 For a book that was co-authored by several authors (up to 10) we have the following change data: what was changed (chapter, section), length of change (number of words), who did the change (author name) and when was the change done (timestamp). Design a visualization approach that helps to explore this data and to detect groups of authors who work together as well as patterns on how they work together. **Explain** your visualization concept, draw sketches to **illustrate** your approach and **justify** your design decisions.

Preliminary Schedule

Thursday, 12-14		
	16.04.2026	Introduction
	23.04.2026	Infographics
	30.04.2026	Visual perception and cognition
	07.05.2026	Visualization of hierarchical data
	14.05.2026	Christi Himmelfahrt
	21.05.2026	Visualization of graphs
	28.05.2026	Visualization of multivariate data and time series
	04.06.2026	Fronleichnam
	11.06.2026	Interaction
	18.06.2026	Software visualization
➔	25.06.2026	Software visualization
	02.07.2026	Some empirical results
	09.07.2026	Exam preparation
	16.07.2026	**Final exam**

Software visualization
Software visualization
Software visualization
Some empirical results & Exam preparation

Chapter 8.4

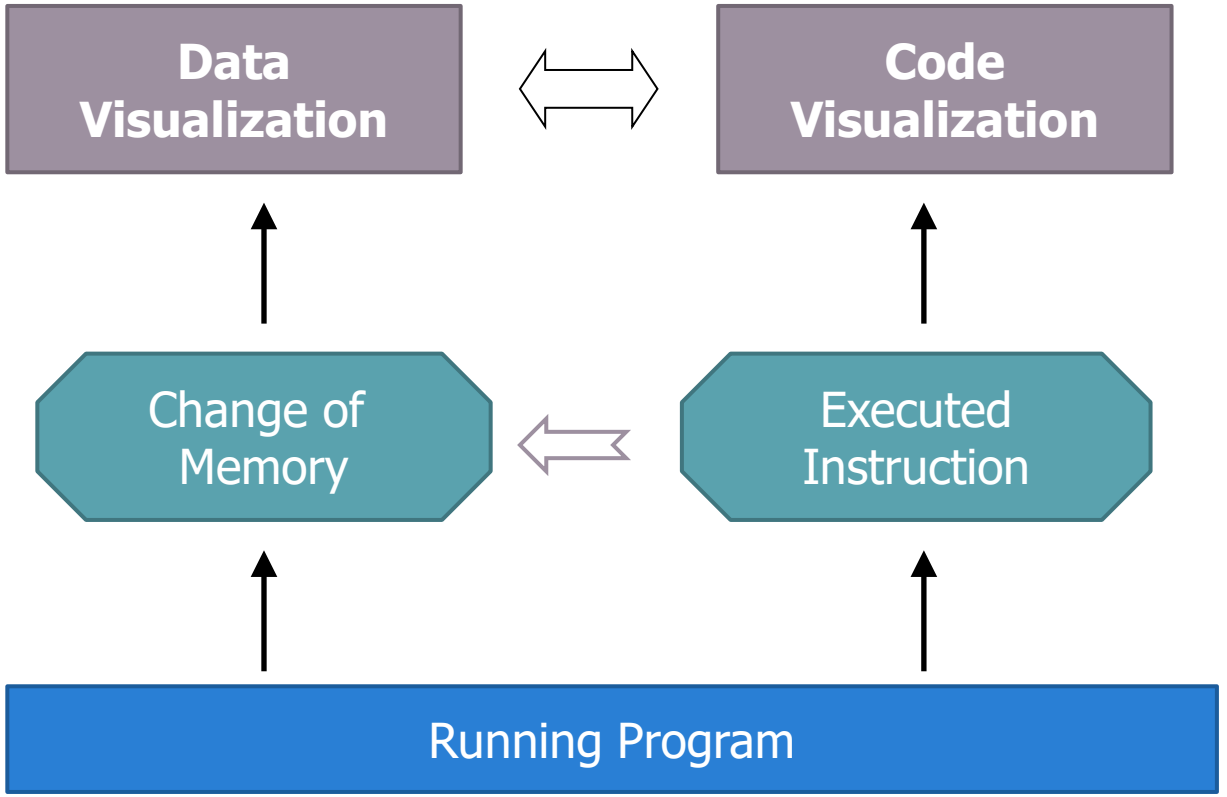
Behavior



5

Dynamic Program Visualization

Dynamic Program Visualization



Dynamic Data Acquisition

Problem: Instrumentation, but also other techniques, are **invasive**, i.e. they influence the runtime behavior.

- Instrumentation
  - before/after each instruction
  - at certain program points
    - before/after loops
    - before/after method calls
    - defined by user (→ interesting events)
  - data structures
    - Whenever data is changed (daemon, observer)

automated vs. manual

source code vs. machine code

- Parallel thread, which reads memory
- Capture messages (for distributed programs)
- Virtual Machine/Interpreter
- Special Purpose Hardware (→ embedded systems)

Dynamic Data Acquisition

What data should be captured?

- Program position (PC, called method, line number in source code)
  - Problem for compiled programs: mapping machine instructions to line numbers in the source code
- Values of program variables
- Heap contents of the program
- For messages:
  - Point in time (enables temporal ordering of messages, which have been captured at different computers, Problem: local vs. global time)

Problem: The more data is captured, the more the program behavior is influenced.

Dynamic Data and Code Visualization

- Accumulative, e.g.
  - DV: Average values of a variable
  - CV: number of times a program point was executed
- Diagram with time axis
  - DV: When was a variable changed
  - CV: When was an instruction executed
- Animation
  - DV: Sequence of diagrams representing the program state at subsequent points in time
  - CV: Displaying the currently executed instruction

```
int m[][]= new int[5][5], m2[][] = new int[5][5];
int old[][]=m, new[][]=m2;
// put a high value in the middle of the matrix
// and all other cells have the initial value 0
m[2][2]=40;

for (int t=0;t<4; t++)
 { for(int i=0; i<m.length; i++)
   { for(int j=0; j<m[i].length; j++)
     { int s=old[i][j];
       int neighbors=1;
       if (i>0) { s=s+old[i-1][j]; neighbors++;}
       if (i<old.length-1) { s=s+old[i+1][j]; neighbors++;}
       if (j>0) { s=s+old[i][j-1]; neighbors++;}
       if (j<old[i].length-1) { s=s+old[i][j+1]; neighbors++;}
       if ((i>0)&&(j>0)) { s=s+old[i-1][j-1]; neighbors++;}
       if ((i>0)&&(j<old[i].length-1)) { s=s+old[i-1][j+1]; neighbors++; }
       if ((i<old.length-1)&&(j>0)) { s=s+old[i+1][j-1]; neighbors++;}
       if ((i<old.length-1)&&(j<old[i].length-1)) { s=s+old[i+1][j+1]; neighbors++;}
       new[i][j]=(int) Math.round(s/(double)neighbors);
     }
   }
 }
```

Example:

Value of a cell is computed by averaging the values of its neighboring cells

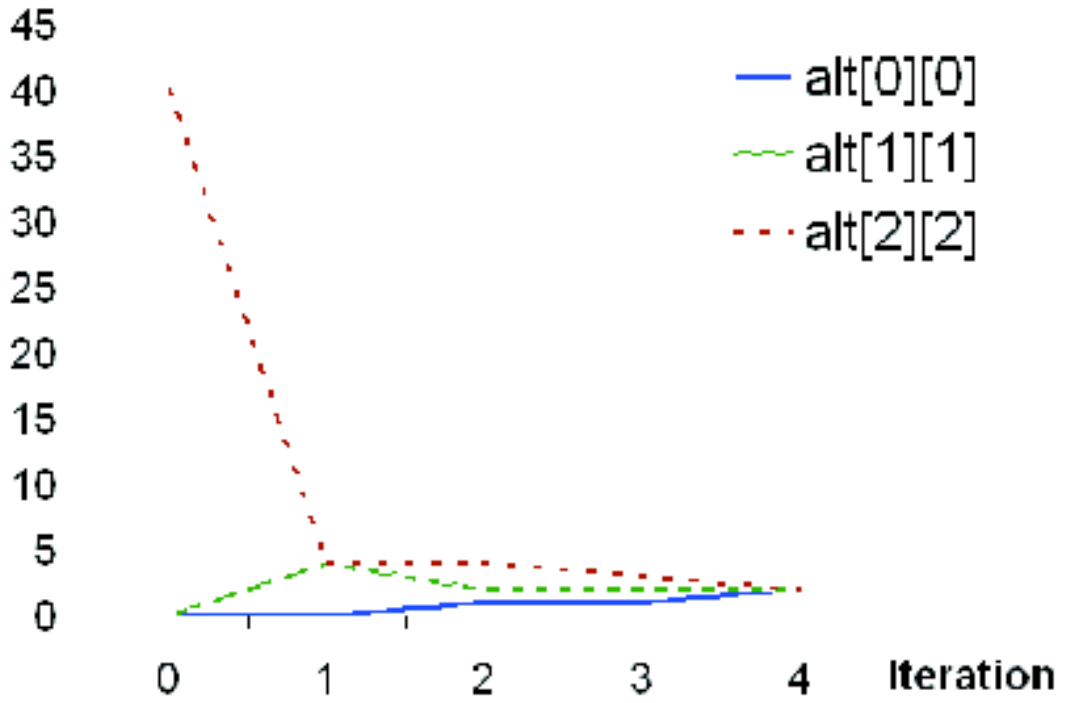
Initial Value	First Iteration	Second Iteration
0 0 0 0 0	0 0 0 0 0	1 1 2 1 1
0 0 0 0 0	0 4 4 4 0	1 2 3 2 1
0 0 40 0 0	0 4 4 4 0	2 3 4 3 2
0 0 0 0 0	0 4 4 4 0	1 2 3 2 1
0 0 0 0 0	0 0 0 0 0	1 1 2 1 1

Third Iteration	Fourth Iteration	Average
1 2 2 2 1	2 2 2 2 2	1 1 1 1 1
2 2 2 2 2	2 2 2 2 2	1 2 2 2 1
2 2 3 2 2	2 2 2 2 2	1 2 11 2 1
2 2 2 2 2	2 2 2 2 2	1 2 2 2 1
1 2 2 2 1	2 2 2 2 2	1 1 1 1 1

Example: Diagram vs. Animation

Initial Value	First Iteration	Second Iteration
0 0 0 0 0	0 0 0 0 0	1 1 2 1 1
0 0 0 0 0	0 4 4 4 0	1 2 3 2 1
0 0 40 0 0	0 4 4 4 0	2 3 4 3 2
0 0 0 0 0	0 4 4 4 0	1 2 3 2 1
0 0 0 0 0	0 0 0 0 0	1 1 2 1 1

Third Iteration	Fourth Iteration	Average
1 2 2 2 1	2 2 2 2 2	1 1 1 1 1
2 2 2 2 2	2 2 2 2 2	1 2 2 2 1
2 2 3 2 2	2 2 2 2 2	1 2 11 2 1
2 2 2 2 2	2 2 2 2 2	1 2 2 2 1
1 2 2 2 1	2 2 2 2 2	1 1 1 1 1





Dynamic Program Visualization

Capturing Data

- Manuel or automatic instrumentation of the source code
- Virtual machine/interpreter

Problem

- Data capturing possibly changes the runtime behavior.

Visualization of temporal change

- aggregated
- time line
- animation

Examples (on the following slides)

- **Algorithm animation**
- **Visual Debugging**

13

Algorithm Animation

Example: Sorting

AlgoRythmics



What algorithm?

https://www.youtube.com/user/AlgoRythmics

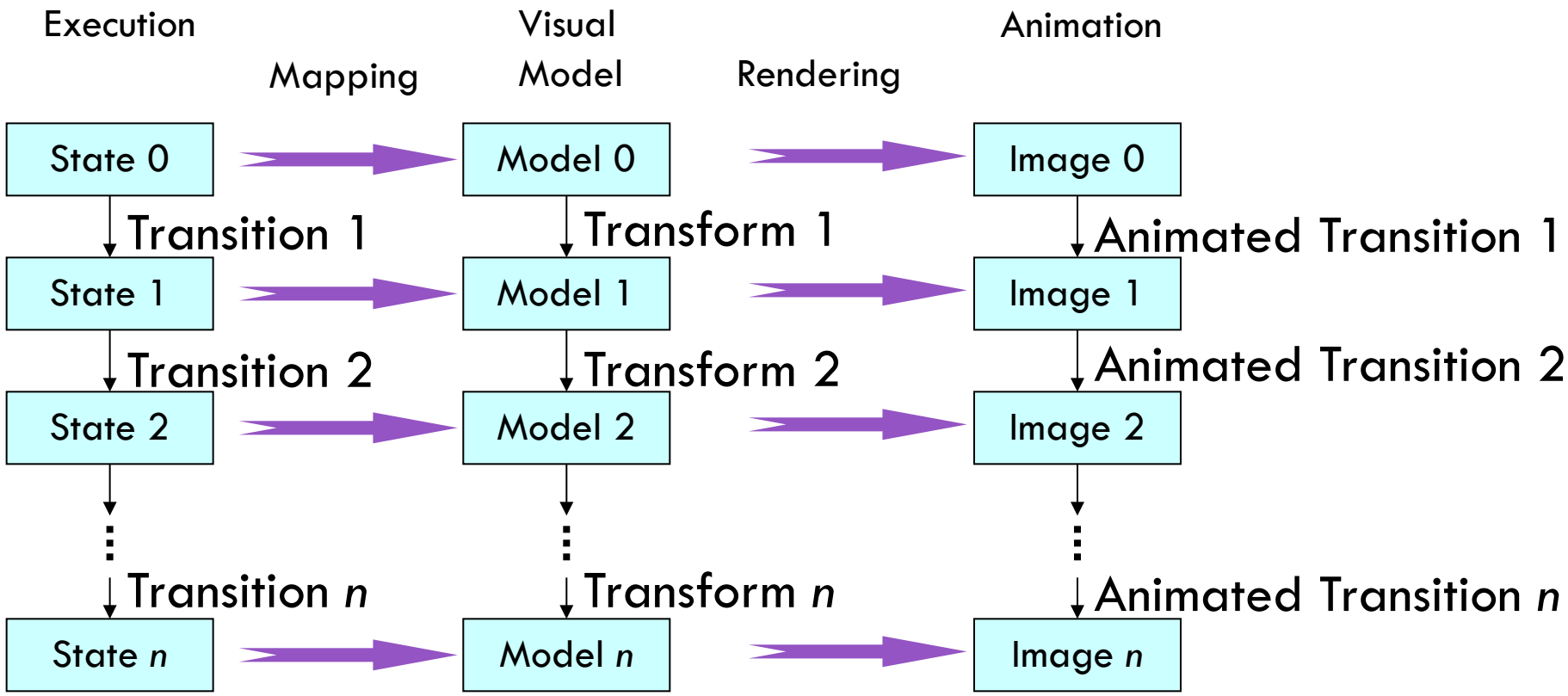
Algorithm Animation

- Algorithm Animation: Visualization of the behavior of an algorithm
- *to animate* = „*to bring to live*“

- Applications
  - Understanding, Teaching
  - Design
  - Optimization
  - Debugging



Model-based Animation

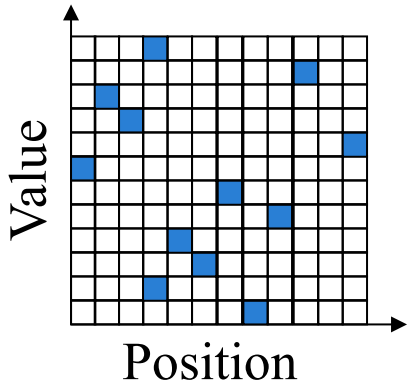


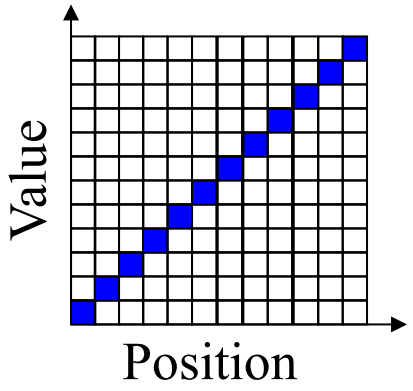
Challenge: Finding the right model

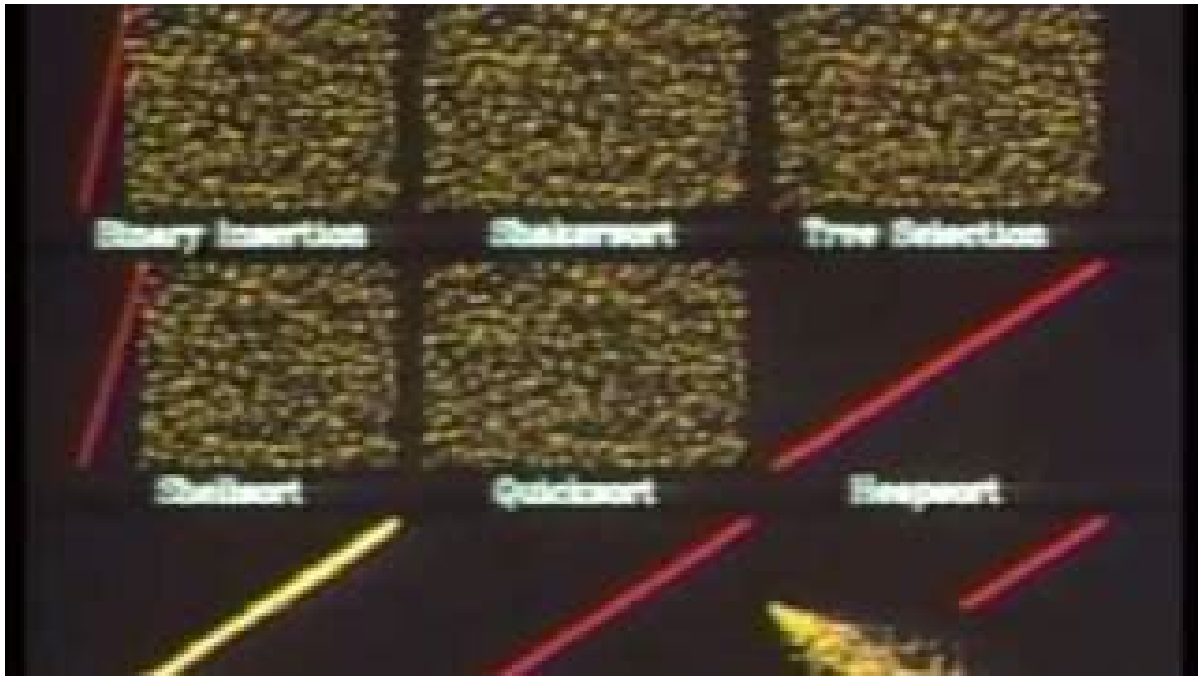
Appropriate graphical abstractions for states and transitions between states.

Sorting Out Sorting

- Manually created film
- Baecker, Ronald M., with the assistance of David Sherman, “Sorting out Sorting”, 30 minute color sound film, Dynamic Graphics Project, University of Toronto, 1981. (Excerpted and “reprinted” in SIGGRAPH Video Review 7, 1983.)



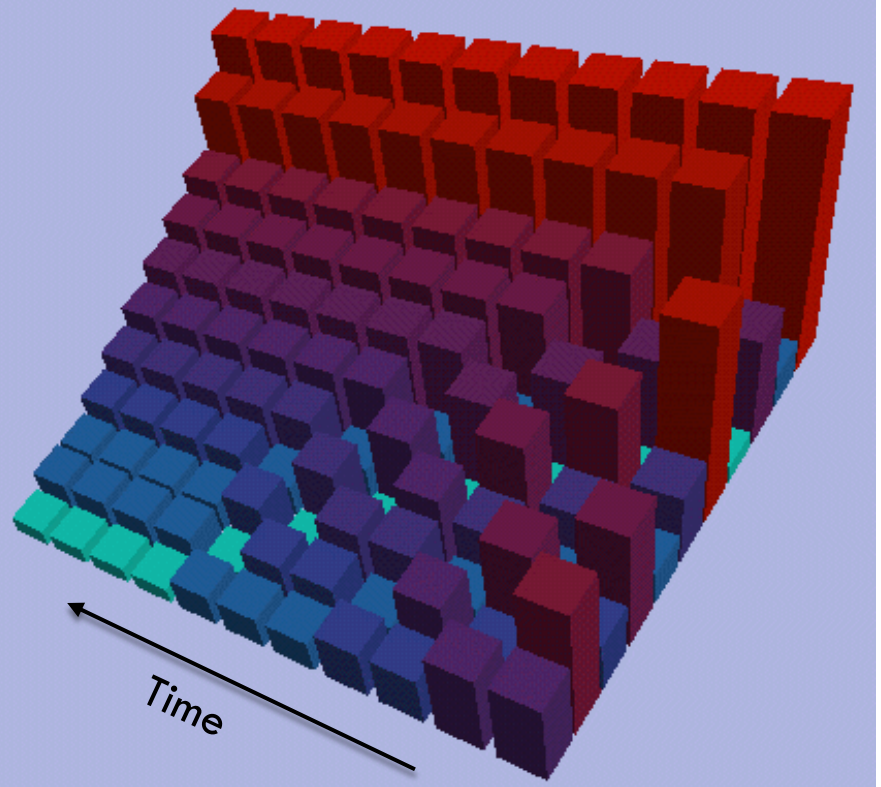




Sorting

- 3rd dimension represents time

5 8 3 7 2 4 11 1 6 2 13
5 3 7 2 4 8 1 6 2 11 13
3 5 2 4 7 1 6 2 8 11 13
3 2 4 5 1 6 2 7 8 11 13
2 3 4 1 5 2 6 7 8 11 13
2 3 1 4 2 5 6 7 8 11 13
2 1 3 2 4 5 6 7 8 11 13
1 2 2 3 4 5 6 7 8 11 13
1 2 2 3 4 5 6 7 8 11 13
1 2 2 3 4 5 6 7 8 11 13
1 2 2 3 4 5 6 7 8 11 13



Architectures for Algorithm Animation

Ad Hoc Visualization and Libraries

- Don‘t use a tool at all. Implement everything **from scratch.**
- Use **libraries** with graphical abstractions, control-elements, etc.

Special datatypes

- Program the algorithm with datatypes which have built-in visualizations

Post-Mortem Visualisierung

- Record an event log or animation script during the execution of the algorithm.
- Animation after the execution of the algorithm.

Interesting Events

- Annotate interesting program points
- Send events to concurrently executed animation (view)

Declarative

- Separation of annotation and algorithm
- Demon monitors state changes and visualizes the state

Semantics-Directed

- automatic visualization by visuall interpreter or debugger for the programming language

Example: Declarative (Leonardo)


```
void main() {
 for(j=n; j>0; j--)
    for(i=1; i<j; i++)
        if (v[i]>v[i+1])
           { int temp= v[i];
              v[i]=v[i+1];
              v[i+1]=temp;
           }
}
```

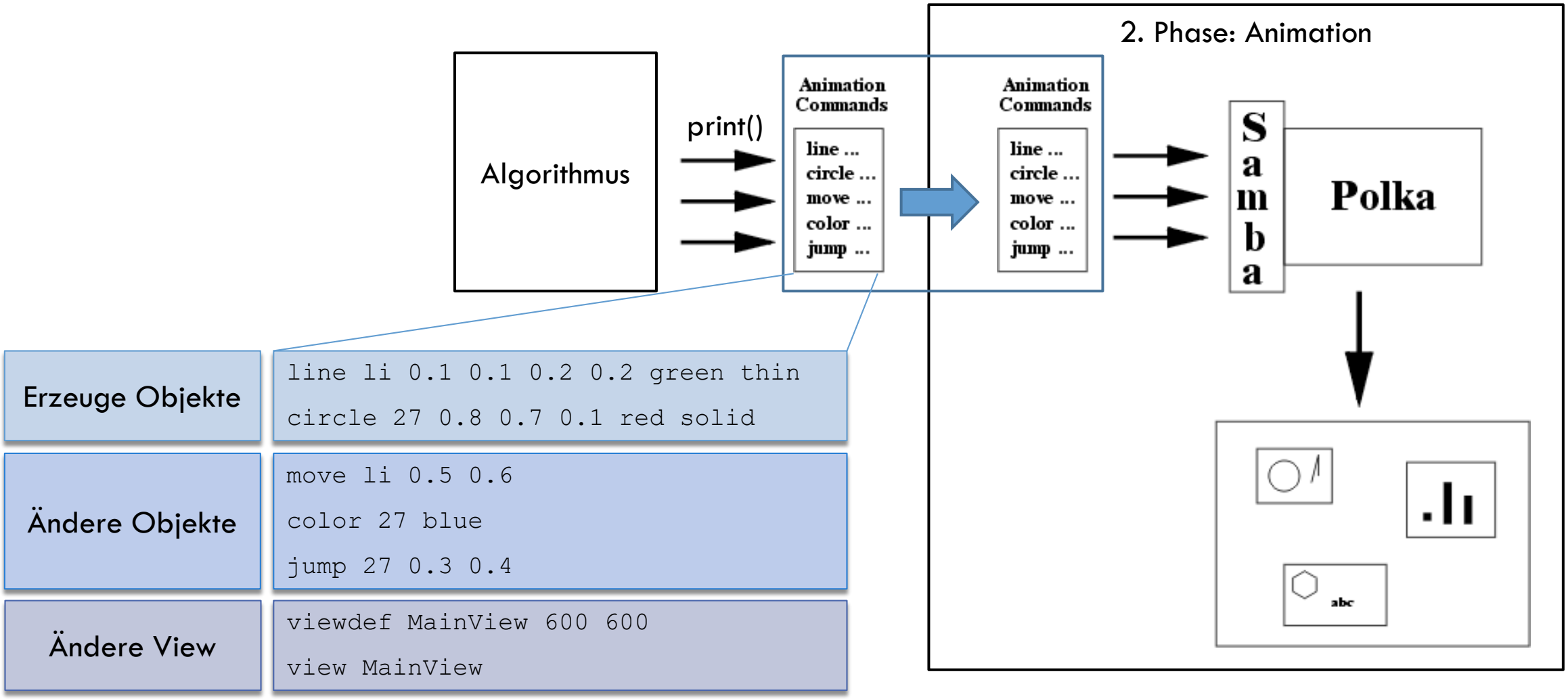
```
/**
View(Out 1);
Rectangle(Out ID, Out X, Out Y, Out L Out H, 1);
    For N: InRange(N,0,n-1)
        Assign X=20+20*N   Y=20   L=15   H=15*v[N]   ID=N;
**/
```

Example: Interesting Events (Polka)


```
void main() {
 bsort.SendAlgoEvt("Input",n,v);
 for(j=n; j>0; j--)
    for(i=1; i<j; i++)
        if (v[i]>v[i+1])
           { int temp= v[i];
             v[i]=v[i+1];
             v[i+1]=temp;
             bsort.SendAlgoEvt("Exchange",i,i+1);
           }
}
```

Bubble Sort

Example: Post-Mortem (Samba)



Erzeuge Objekte	`line li 0.1 0.1 0.2 0.2 green thin` `circle 27 0.8 0.7 0.1 red solid`

Ändere Objekte	`move li 0.5 0.6`
	`color 27 blue`
	`jump 27 0.3 0.4`

Ändere View	`viewdef MainView 600 600` `view MainView`

Visual Debugging

Debugging

- Debugging = Detecting, locating and fixing errors in programs
- Interactive Debugger



- Application
  - Find incorrect data (program state / memory)
  - Find erroneous statements

GNU Data Display Debugger (DDD)

- GUI for GNU command line debugger
- Visualization of program state

```
1: *tree
  value        = 7
  _name        = 0x8049e88 "Ada"
  _left        = 0x804d7d8
  _right       = 0x0
  left_thread  = false
  right_thread = false
                 day_of_week = Thu
                 day         = 1
  date         = month       = 1
                 year        = 1970
                 _vptr.      = 0x8049f78 <Date virtual table>
  shared       = 4711
```

(gdb) **display *tree**

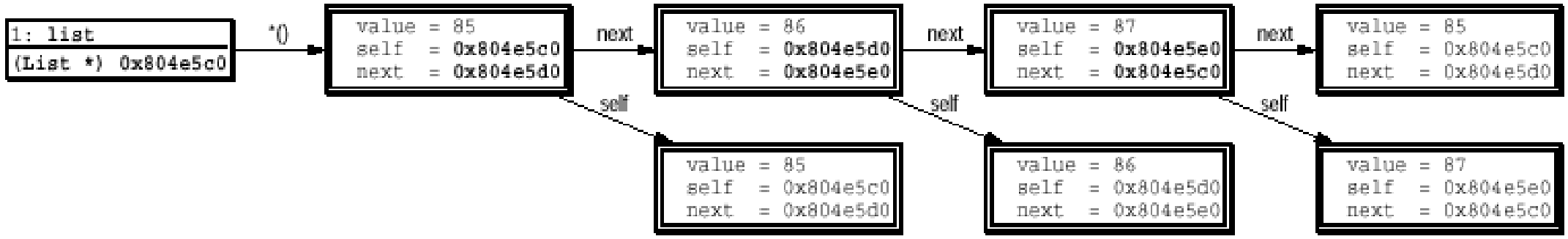
*tree = { fvalue = 7, _name = 0x8049e88 "Ada",
  _left = 0x804d7d8, _right = 0x0,
  _left_thread = false, right_thread = false,
  date = {day of week = Thu, day = 1, month = 1,
          year = 1970, _vptr. = 0x8049f78 <Date virtual table>},
  static shared = 4711}

(gdb) _

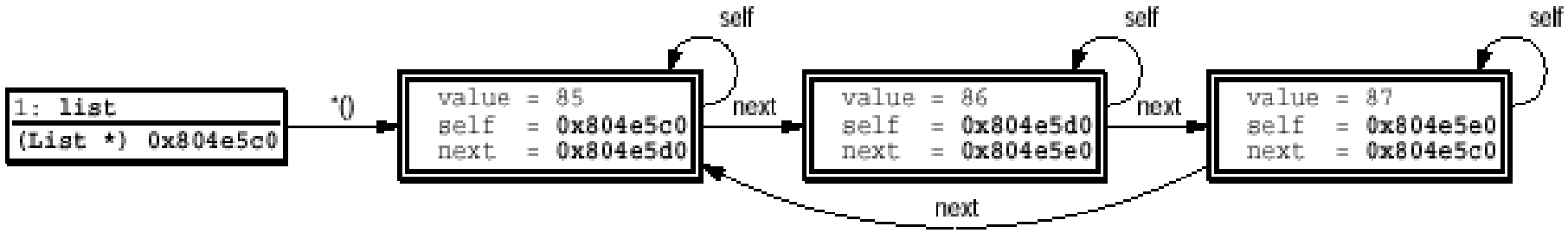
Zeller und Lütkehaus, 1996

GNU Data Display Debugger (DDD)

- Interactive unfolding of data structure



- Merge nodes with same address (alias detection)

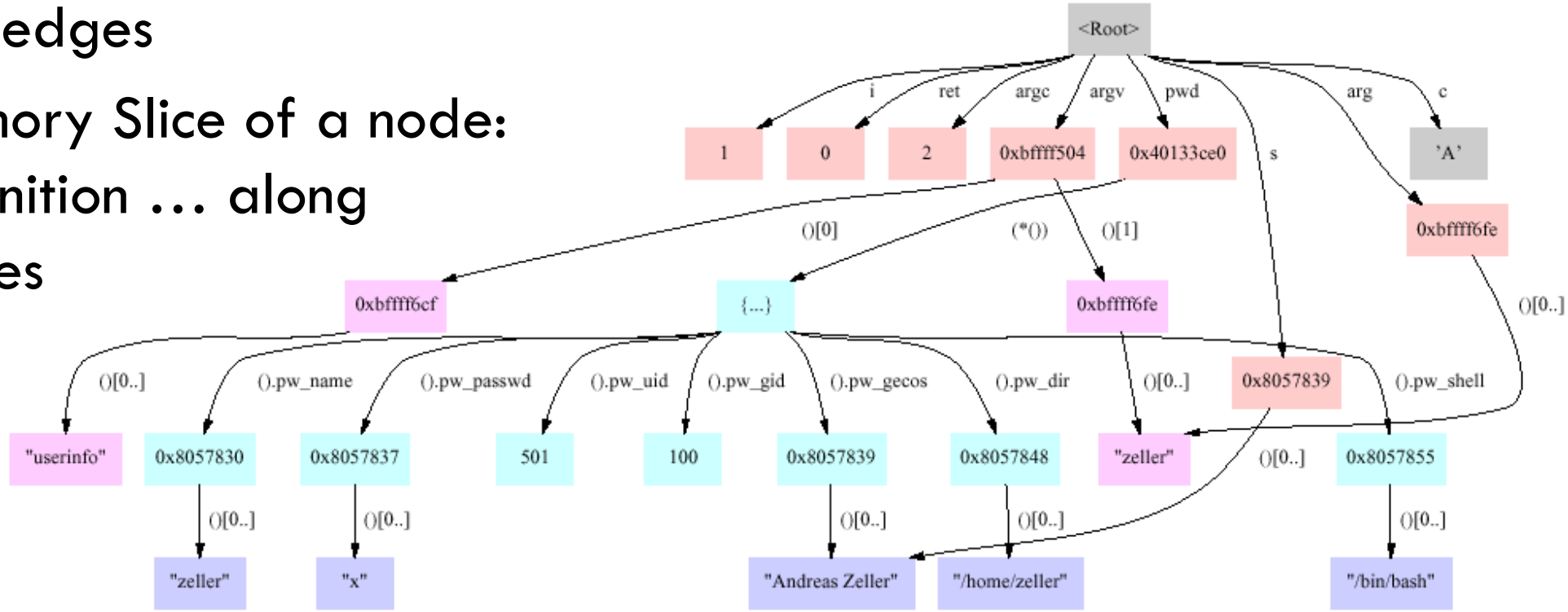


Memory Graphs

- Representation of the memory of a program.
- *Nodes* = memory content
- *Arrows* = possible access paths.
- unfolding **all** accessible data structures in the program. All common data structures like structs, unions, arrays or pointers are properly represented.
- Memory graphs may become very large.
- Visualization
  - Focus on small subgraphs
  - Focus on changes of the graph
  - Aggregation of recurring patterns → e.g. Reference Patterns

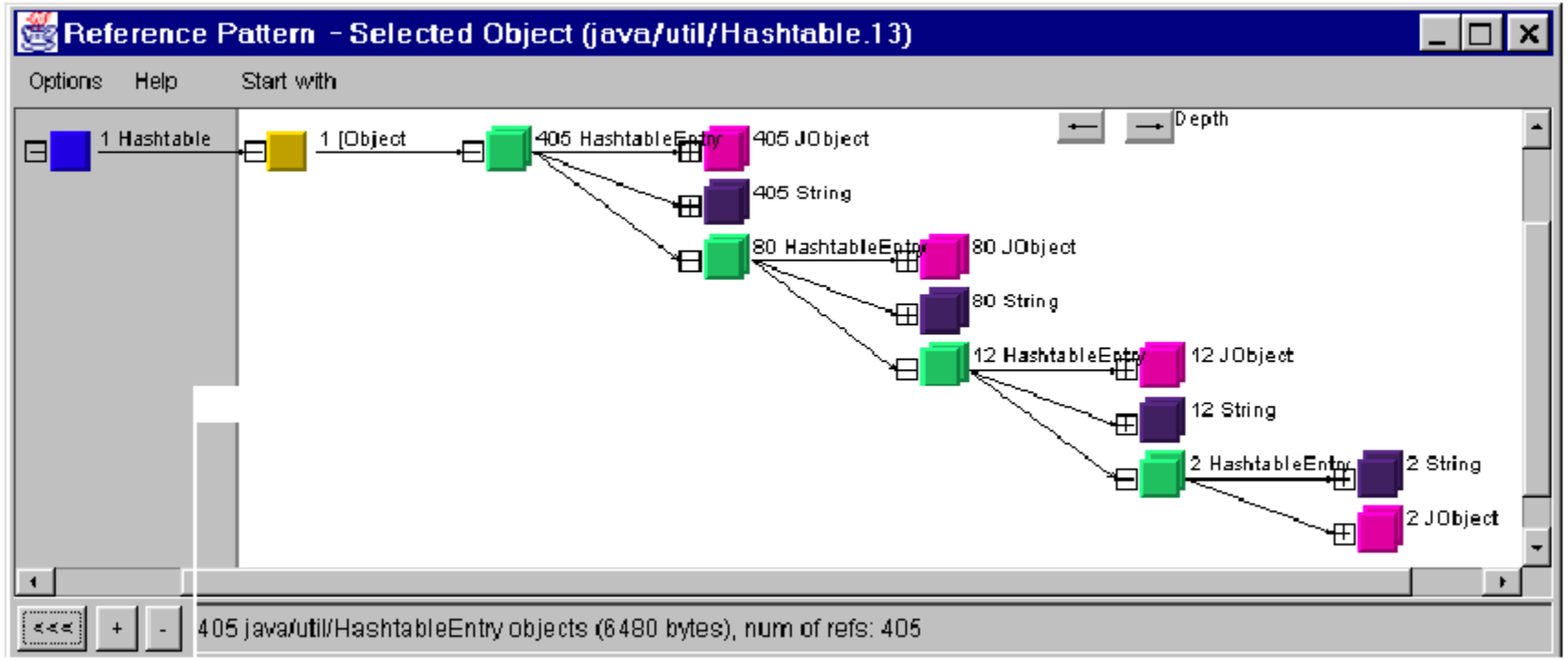
Memory Slices

- Forward Memory Slice of a node: subgraph reachable from the node along forward edges
- Backward Memory Slice of a node: analogous definition … along backward edges



Extraction of reference patterns
Jinsight (http://www.research.ibm.com/jinsight/)

- At each level/depth objects of the same class are represented by a single node



Extra slides – not covered in lecture !

Reference Pattern

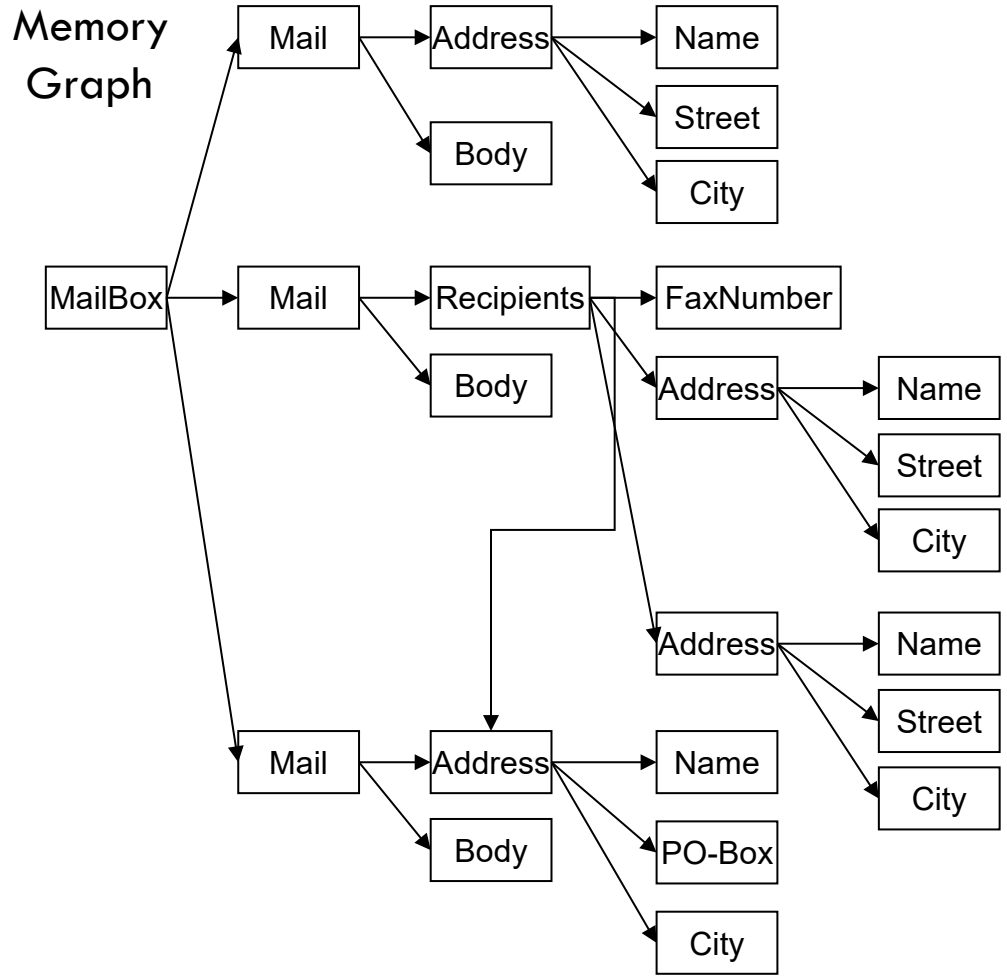
- Aggregation of memory graphs

  → reference patterns are trees

- The root of the tree represents all objects of a single class of the start set.
- Each other node represents all objects of a single class, which are referenced by an object represented by the parent node.

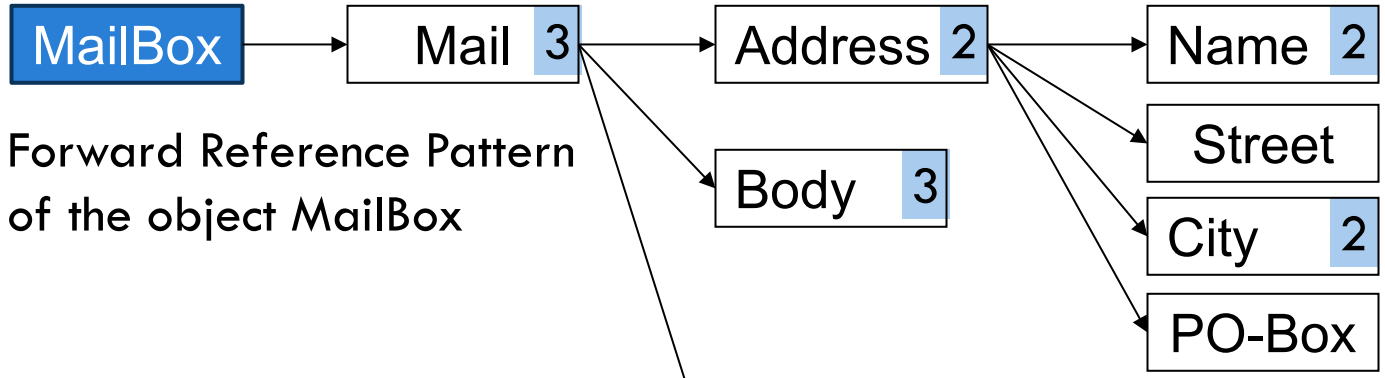
Extra slides – not covered in lecture !

Example: Forward Reference Pattern



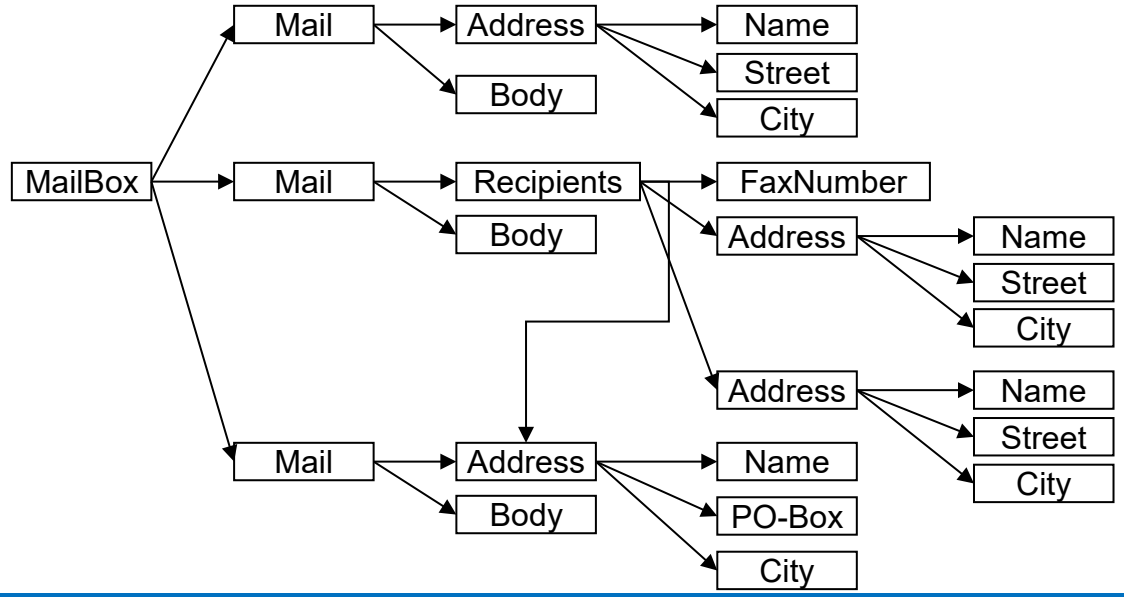
Extra slides – not covered in lecture !

Example: Forward Reference Pattern



Forward Reference Pattern of the object MailBox

Memory Graph



Extra slides – not covered in lecture !

Forward Reference Pattern

More precisely, a reference pattern is a tuple $(V, E, root)$ where the nodes in in V have the form (O, t, d) with:

- O is a set of objects of the same type t
- d is the depth of this node, i.e. the length of the path from the root $root$ to this node

Given a forward memory slice $S_F(v) = (V, E, root)$ we define a forward reference pattern $R_F(v) = (V', E', root')$ of the variable v as follows:

- $root' = (\{o\}, t, 0)$ where o is the object referenced by v and t its type
- $root' \in V$
- if $(O, t, d') \in V'$ and $d' < d$ then

$$\begin{cases} ((O,t,d'),(O_i,t_i,d+1)) \in E' & \text{if } t \text{ is an object type and } O_i = \{o' \mid (o,v'),(v',o') \in E, o \in O, v' \text{ is a variable, and the type of } o' \text{ is } t_i\} \\ ((O,t,d'),(O',t',d+1)) \in E' & \text{if } t \text{ is an array type and } O' = \{o' \mid (o,o') \in E, o \in O, \text{and the type of } o' \text{ is } t'\} \end{cases}$$

Extra slides – not covered in lecture !

Backward Reference Pattern

Analogously, given a backward memory slice $S_B(o) = (V, E, root)$, we define a backward reference pattern $R_B(o) = (V', E', root')$ of the object o as follows:

- $root' = (\{o\}, t, 0)$ where t is the type of the object o,
- $root' \in V$,
- if $(O, t, d) \in V'$

$$\begin{cases} ((O,t,d),(O_i,t_i,d+1)) \in E' & \text{where } O_i = \{o' | (o', o'.v), (o'.v, o) \in E, \\ & \quad o \in O,\ v \text{ is an object variable,} \\ & \quad \text{and the type of } o' \text{ is } t_i\}, \\ ((O,t,d'),(O',t',d+1)) \in E' & \text{where } O' = \{o' | (o', o) \in E,\ o \in O, \\ & \quad \text{and the type of } o' \text{ is an array type } t'\}. \end{cases}$$

Because of cycles, reference patterns can become infinite, and thus in practice we only compute them up to some given depth.

Extra slides – not covered in lecture !

Slicing & Dicing

- **Static Slice**
  - set of all program points that **may** affect the value of a particular output or an instance of a variable at a certain program point.
  - → data flow analysis
- **Dynamic Slice**
  - set of all program points that for a given input **actually affect** a program point or instance of a variable at a certain program point.
- **Execution Slice**
  - set of all program points executed for a given input.
- **Dice**
  - set difference of two slices

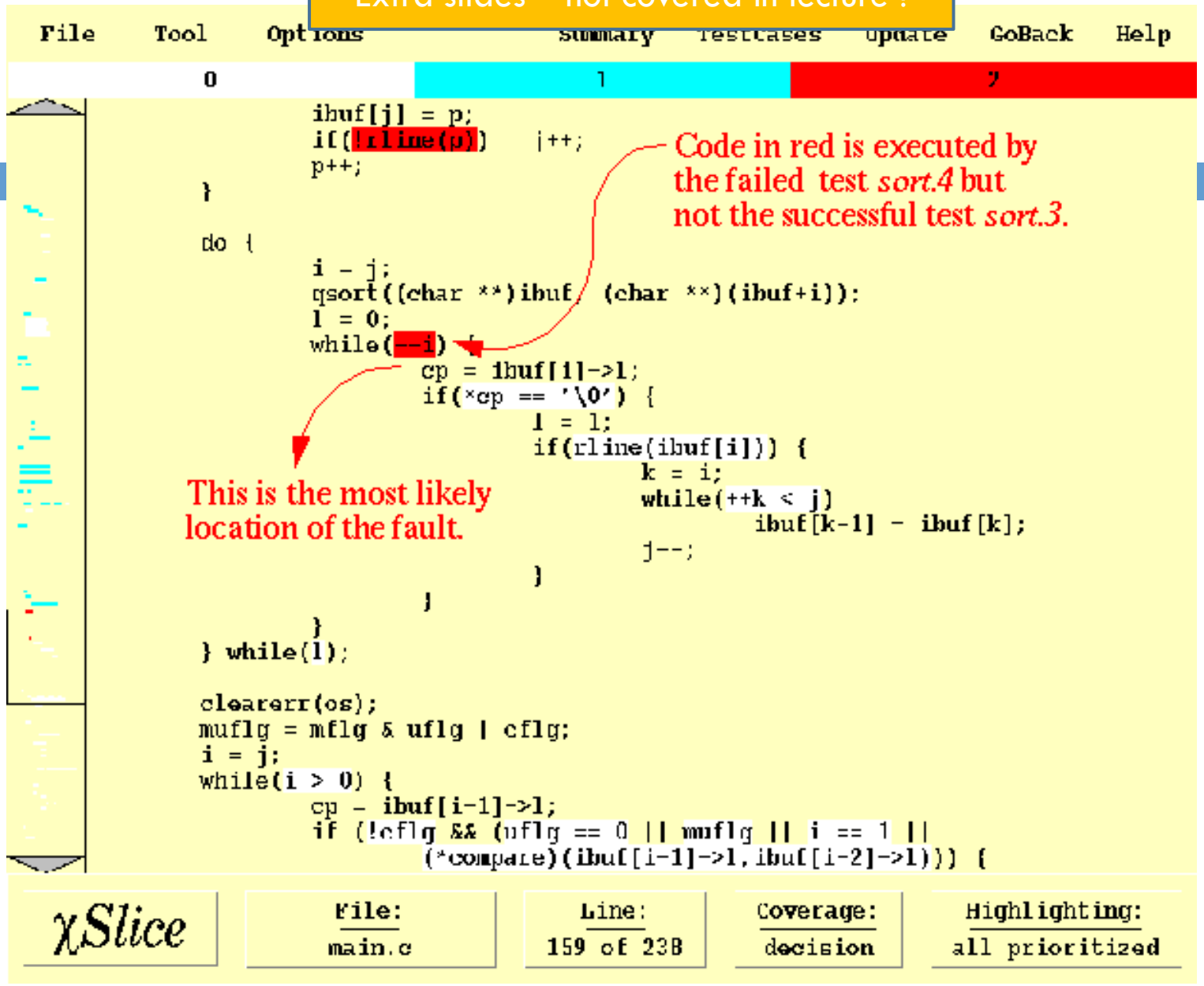
Test Coverage

- Test a progam with automated test cases (e.g. Junit)
- Test coverage of statements
  - What test executed which statements?
- For debugging purposes: dividing test cases into
  - successful (passing)
  - failing

Extra slides – not covered in lecture !

X-Slice

- X-Slice is a slicing and dicing tool for C programs.
- the dice represents the intersection of a failing test case and a sucessful test case.



Extra slides – not covered in lecture !

Color coding of coverage data

„probability“

$$\text{hue}(s) = \text{low hue (red)} + \frac{\%\text{passed}(s)}{\%\text{passed}(s)+\%\text{failed}(s)} * \text{hue range}$$

$$\text{bright}(s) = \max(\%\ \text{passed}(s), \%\ \text{failed}(s))$$

„evidence“

%passed(s): ratio of successful test cases, which cover a statement

%failed(s): analogous for failing test cases

What does this color code mean?

Example:

- Buggy computation of leap years

- Hue indicates probability

- Brightness the amount of evidence

	year	1800	1998	2000	2002	2004	2006	2008	passed	failed	%passed	%failed	hue	brightness
1	int isLeapYear(int year)													
2	{ if (y%4==0)	1	1	1	1	1	1	1	4	3	1,00	1,00	0,50	1,00
3	if (y%100==0)	1	0	1	0	1	0	1	1	3	0,25	1,00	0,20	1,00
4	if (y%400==0)	1	0	1	0	0	0	0	1	1	0,25	0,33	0,43	0,33
5	return true;	0	0	1	0	0	0	0	1	0	0,25	0,00	1,00	0,25
6	else return true;	1	0	0	0	0	0	0	0	1	0,00	0,33	0,00	0,33
7	return false; }	0	1	0	1	1	1	1	3	2	0,75	0,67	0,53	0,75
	Result:	true	false	true	false	false	false	false						
	Passed:	no	yes	yes	yes	no	yes	no						

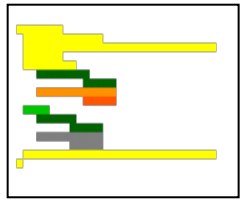
Missing braces

Every year that is exactly divisible by four is a leap year, except for years that are exactly divisible by 100, but these centurial years are leap years if they are exactly divisible by 400.

Tarantula

Jones et al., 2001


```
mid() {
  int x,y,z,m;
  read("Enter 3 numbers:",x,y,z);
  m = z;
  if (y<z)
     if (x<y)
           m = y;
     else if (x<z)
           m = y;
  else
     if (x>y)
           m = y;
     else if (x>z)
           m = x;
  print("Middle number is:", m);
}
```





Summary

Dynamic Programm Visualization

- Capturing the program behavior (Memory, Instructions)
- Visualization: aggregated, time line, animation

Example: Algorithm Animation

- Facilitates comprehension and optimization of algorithms
- Various different architectures

Example: Visual Debugging

- Data: DDD, Memory Slices, Reference Patterns
- Statements: Slicing & Dicing, Test Coverage

Chapter 8.5

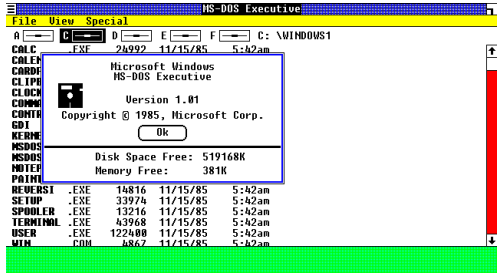
Software Evolution

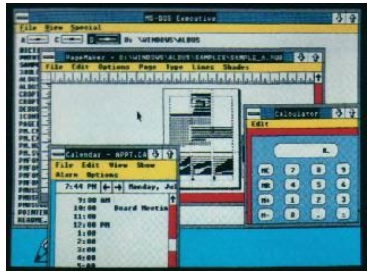


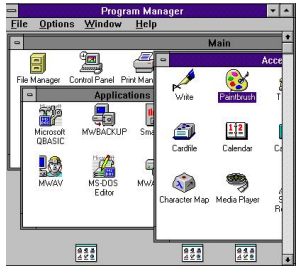
What is software evolution ?

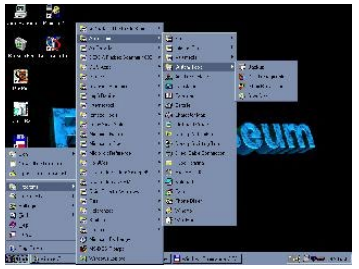
- „Software Evolution“ is a synonym for „Software Development Process“ emphasizing that a software product changes throughout its „lifetime“.

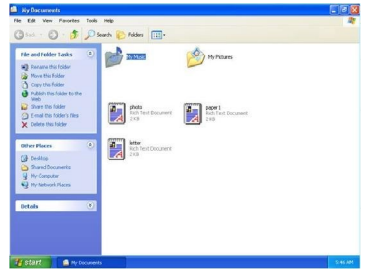
Windows 1.01 Windows 2.0 Windows 3.0 Windows 95 Windows XP







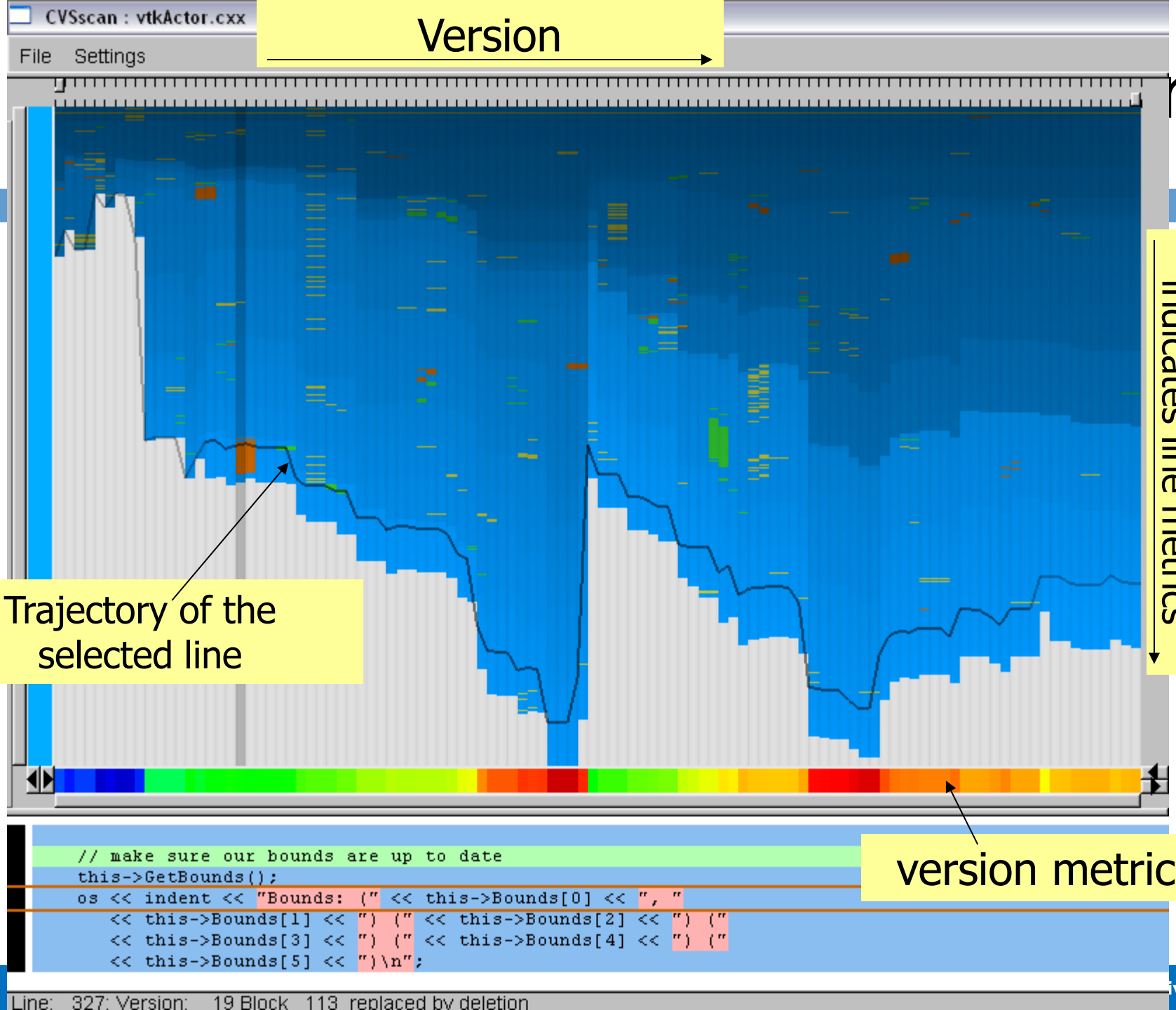




"Software aging will occur in all successful products."

[David L. Parnas, 1994]

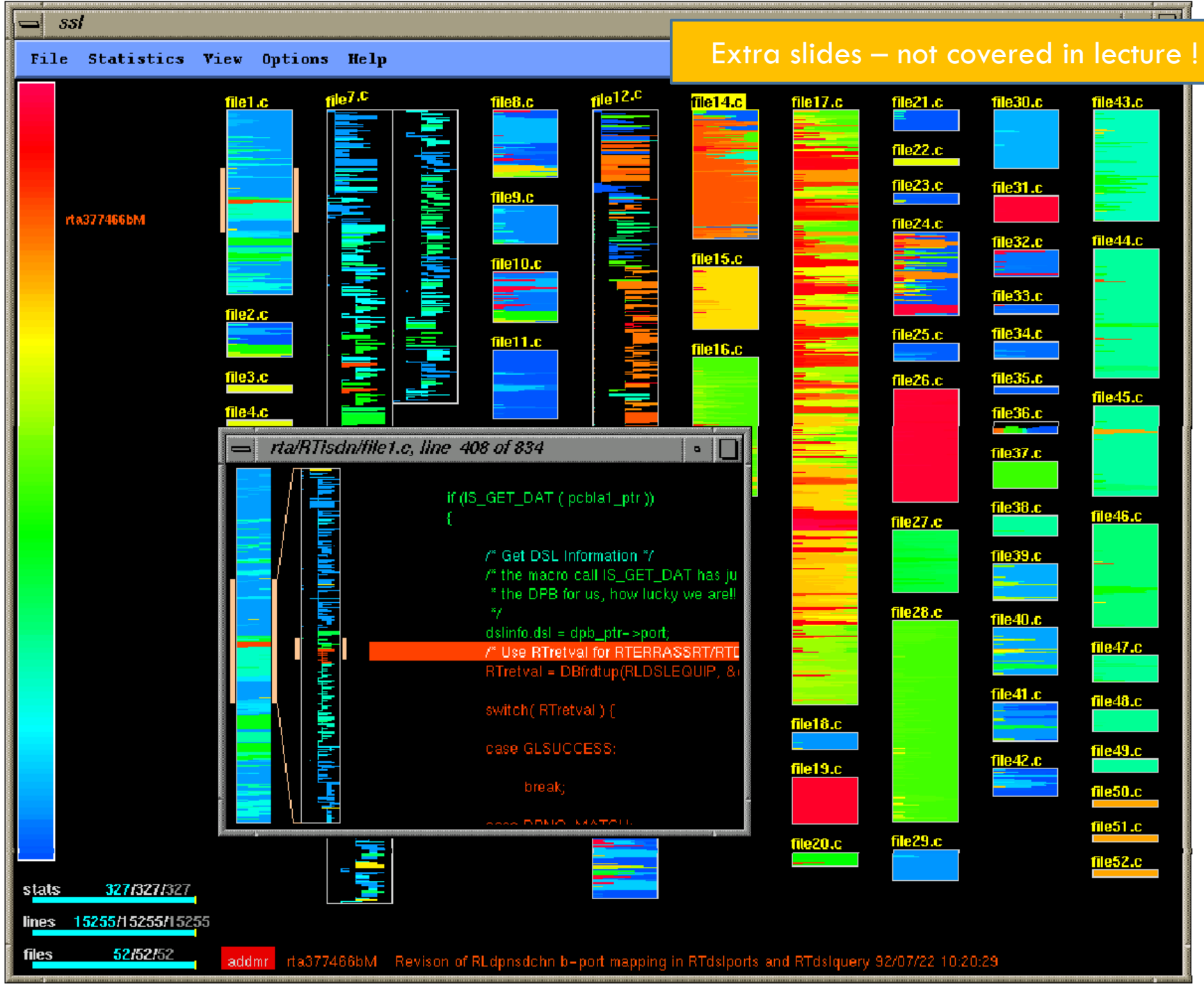
Evolution on the level of lines of code



CVSscan

Extra slides – not covered in lecture !

Evolution on the level of lines of code

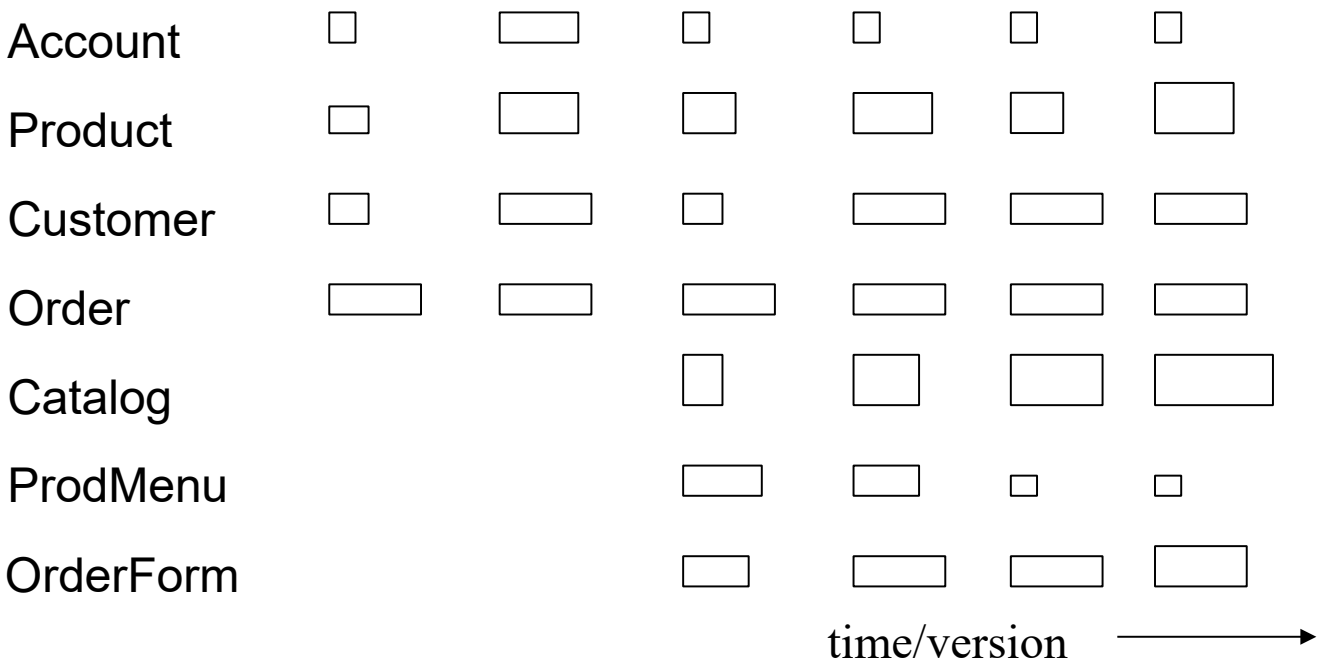


SeeSoft

Color indicates time of most recent change of line of code (= code age).

Evolution Matrix

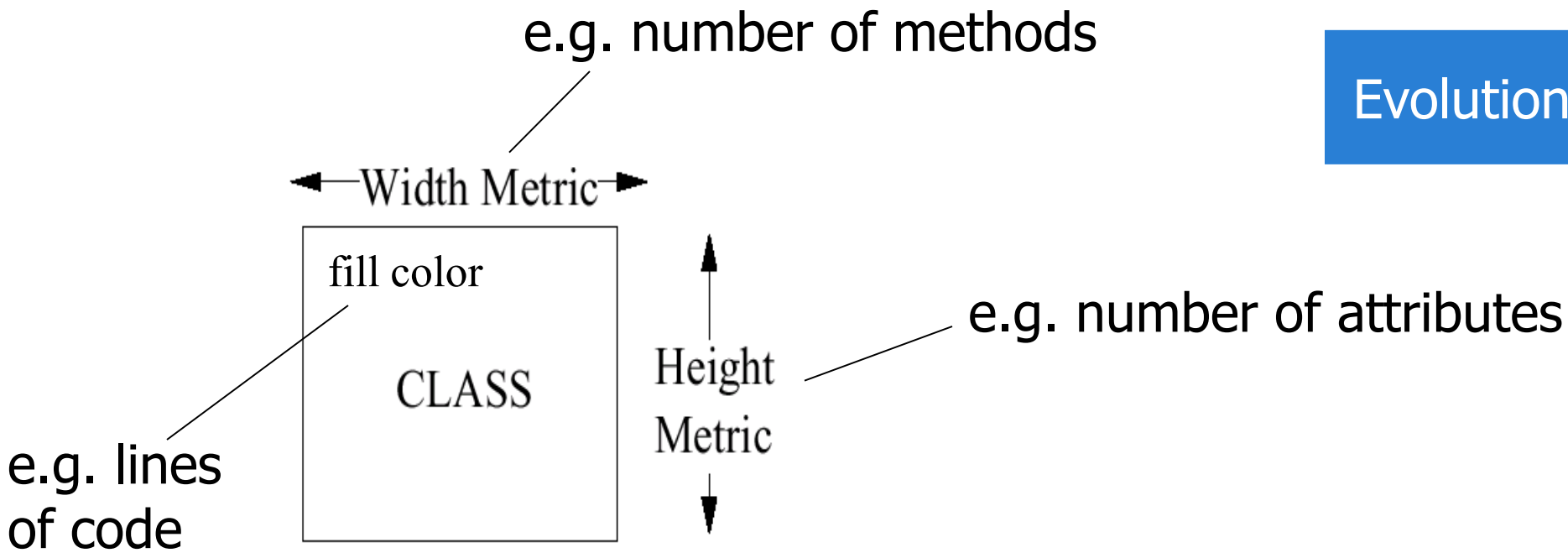
- Visualization of the evolution of an object-oriented system, i.e. a set of classes



Evolution Matrix

Source: **The Evolution Matrix: Recovering Software Evolution using Software Visualization Techniques.** Michele Lanza *Michele Lanza, 4th International Workshop on Principles of Software Evolution (IWPSE '01), ACM,.2001.. https://doi.org/10.1145/602461.602467*

Representation of a class



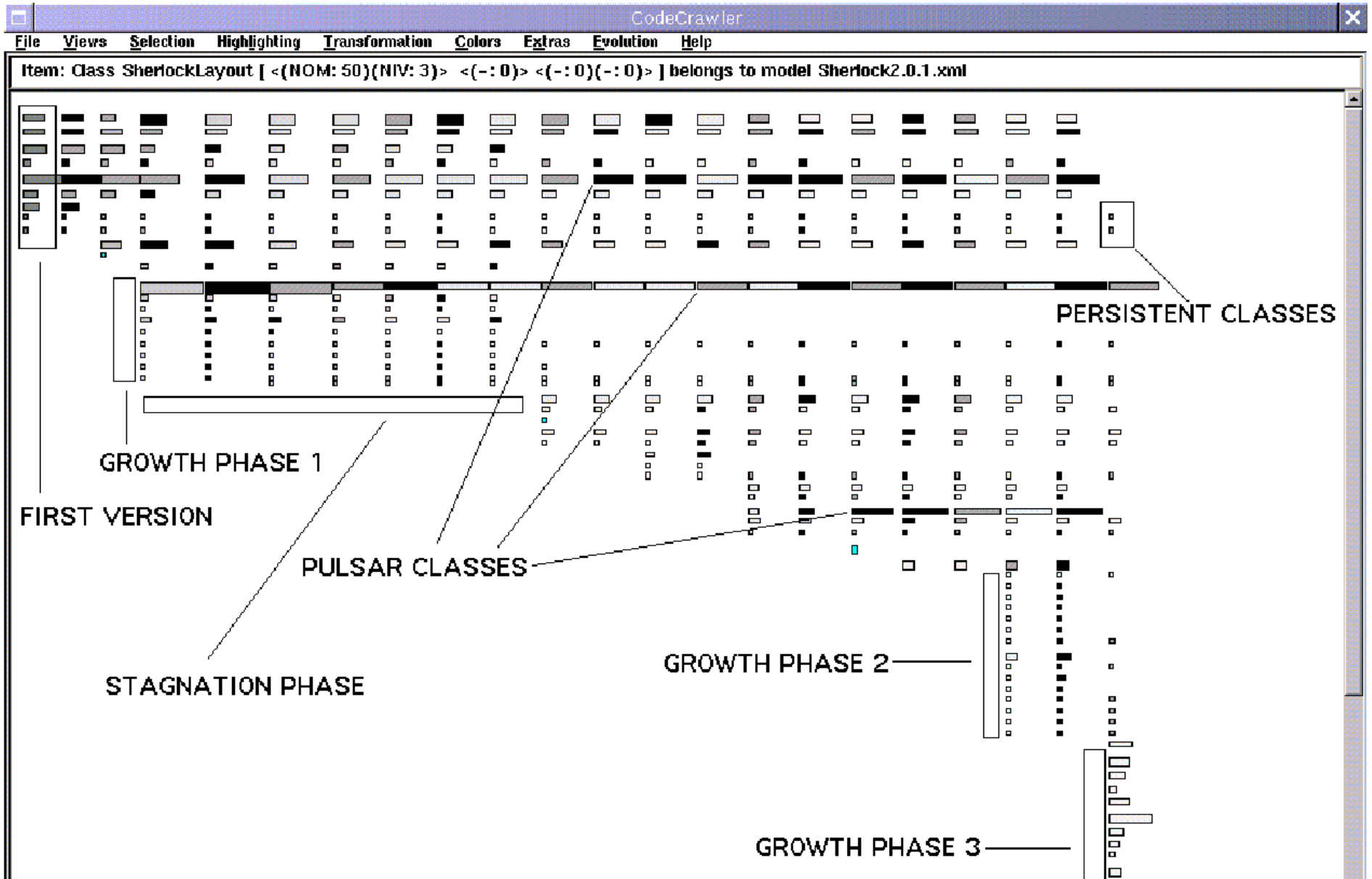
Evolution Matrix

The visualization approach to use size and color of boxes to indicate software metrics is aka **Polymetric View**

Extra slides – not covered in lecture !

Example:

Evolution on the level of classes



Evolution Matrix

Observerations about the whole system

Extra slides – not covered in lecture !

Evolution Matrix

- System Size
  - Adding and Removing Classes



  - Phases of Growth, Stagnation and Shrinking

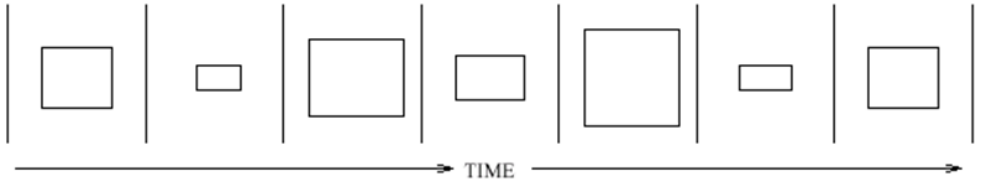
Categorizing individual classes

Extra slides – not covered in lecture !

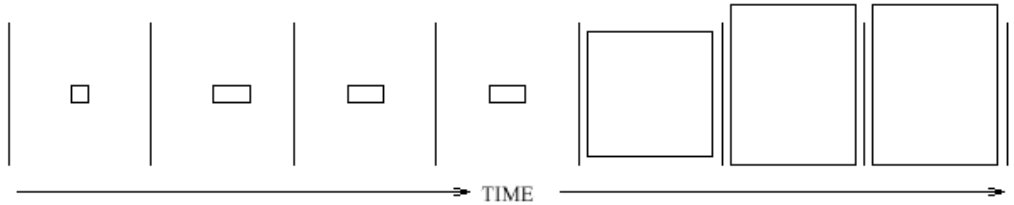
Evolution Matrix

- Pulsar
- Supernova
- White Dwarf
- Red Gigant
- Dayfly
- Persistent
- Stagnant

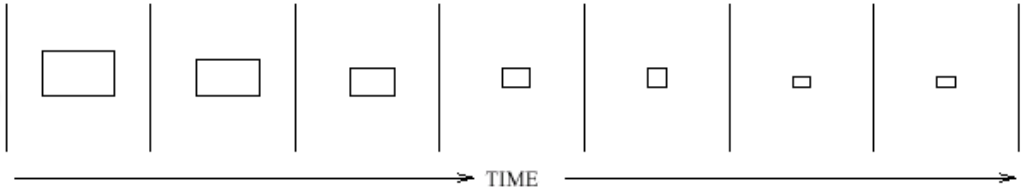
Note that the claims have not been sufficiently evaluated by the authors!



Implementation of new functionality increases, while refactoring decreases size of class. Class may be a hotspot in the system.



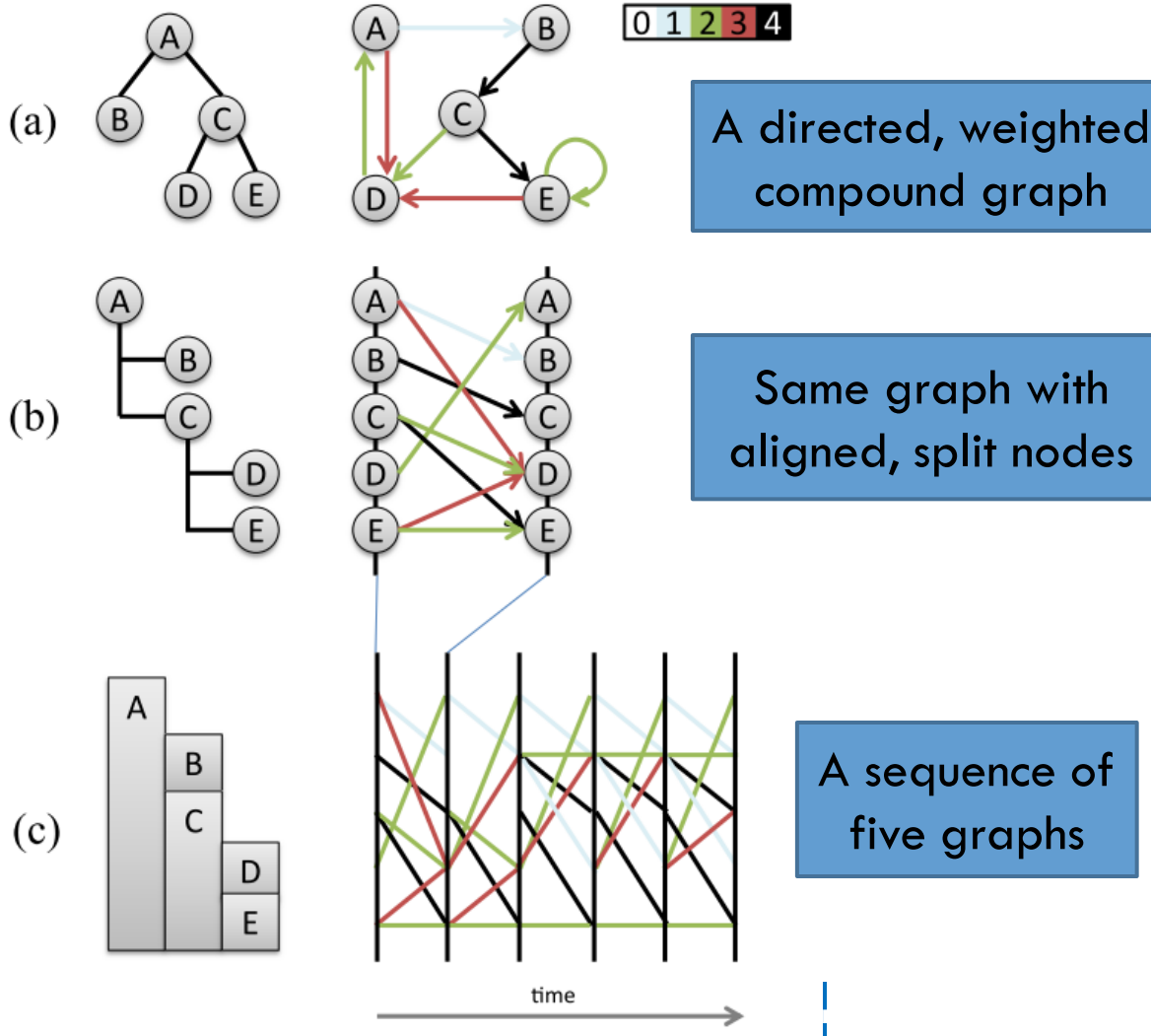
Three common cases: massive refactoring, data class (lots of constants), or sleeper class (defined a long time ago and waiting to be filled with functionality). The accelerated growth rate may be a sign of unclean design or introduce new bugs.



Functionality was moved to other classes. Class may have become obsolete.

Visualizing the evolution of call graphs

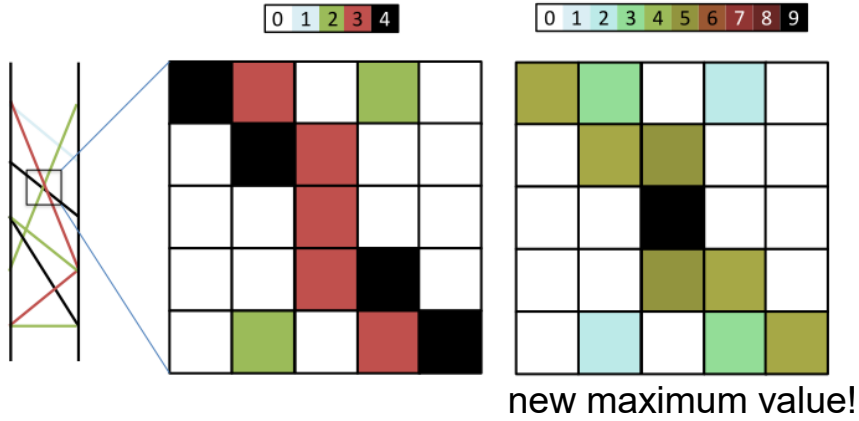
- Parallel Edge Splatting for Visualizing Dynamic Graphs

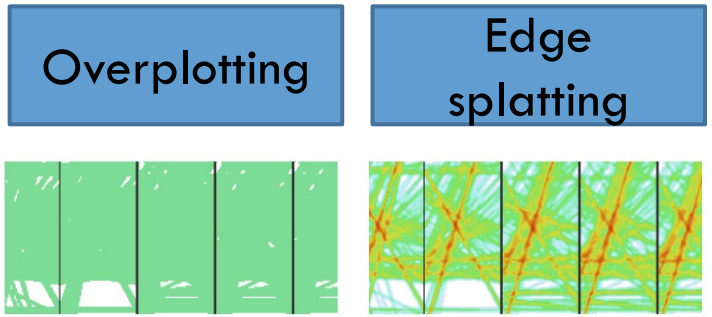


A directed, weighted compound graph

Same graph with aligned, split nodes

A sequence of five graphs

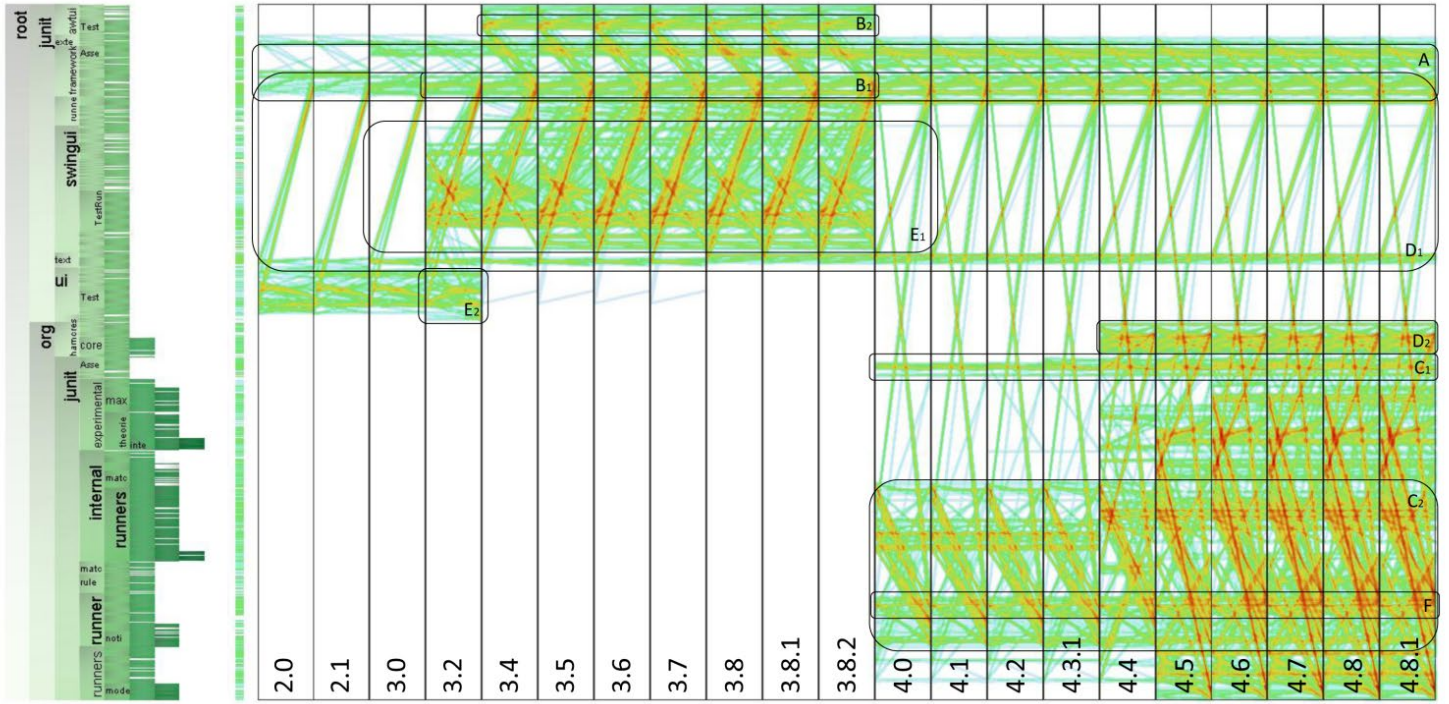




Visualizing the evolution of call graphs

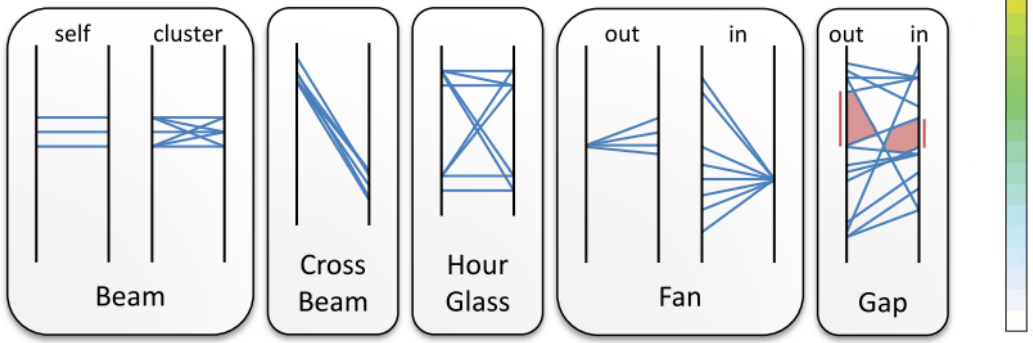
- Parallel Edge Splatting for Visualizing Dynamic Graphs

Call dependencies between methods in 21 releases of the JUnit project: all(top), added (left), and removed (right) dependencies.



Source: **Parallel Edge Splatting for Scalable Dynamic Graph Visualization**. M. Burch, C. Vehlow, F. Beck, S. Diehl, and D. Weiskopf. *IEEE Transactions on Visualization and Computer Graphics* 17, 12, 2011

Visual Patterns



Visual Data Mining in Software Archives

Learning from the evolution of software systems

Extra slides – not covered in lecture !

Software Archives

- **Configuration Management Systems** like RCS, CVS, SVN, or GIT keep track of versions and changes of the software related documents (code, scripts, documentation, …)
- Other tools store additional information, e.g. **bug or issue tracking tools**.
- The term **software archive** refers to the collection of information stored by these tools. It contains the development **history** of the system.

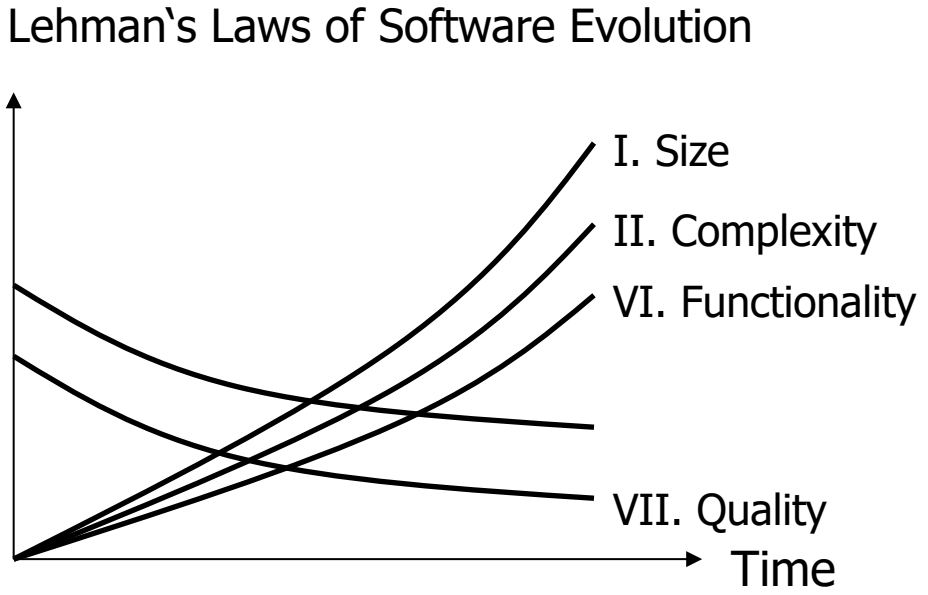
Observation

- We presented different tools and techniques for visualizing the change of the structure, metrics and source code of a system over time.

- There is much more information in software archives than these tools exploit, because they leave the exploration and analysis to the user.

Problem of published rules and „laws“

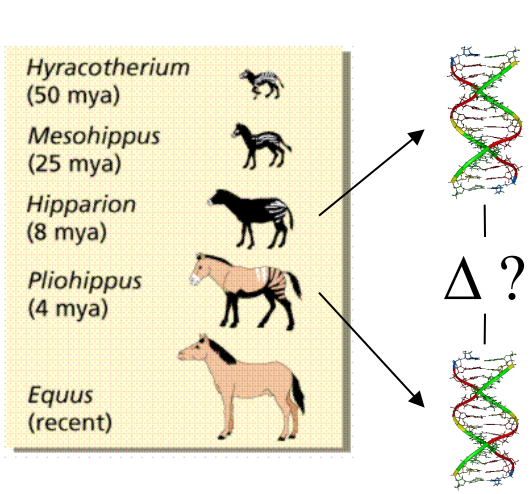
- Relevance of laws and case studies for your own project
  - „Laws“ are too general
  - Insights of case studies are difficult to transfer

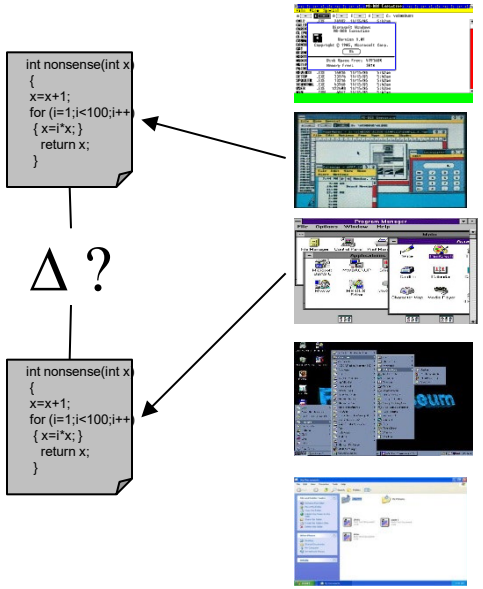


⇒ Tools, that learn (actionable) rules from your own previous or current projects.

Scientific approach in analogy to biology

- Search for recurring patterns in differences
- Formulate rules
- Validate rules with historical data
- Apply rules to predict the future.



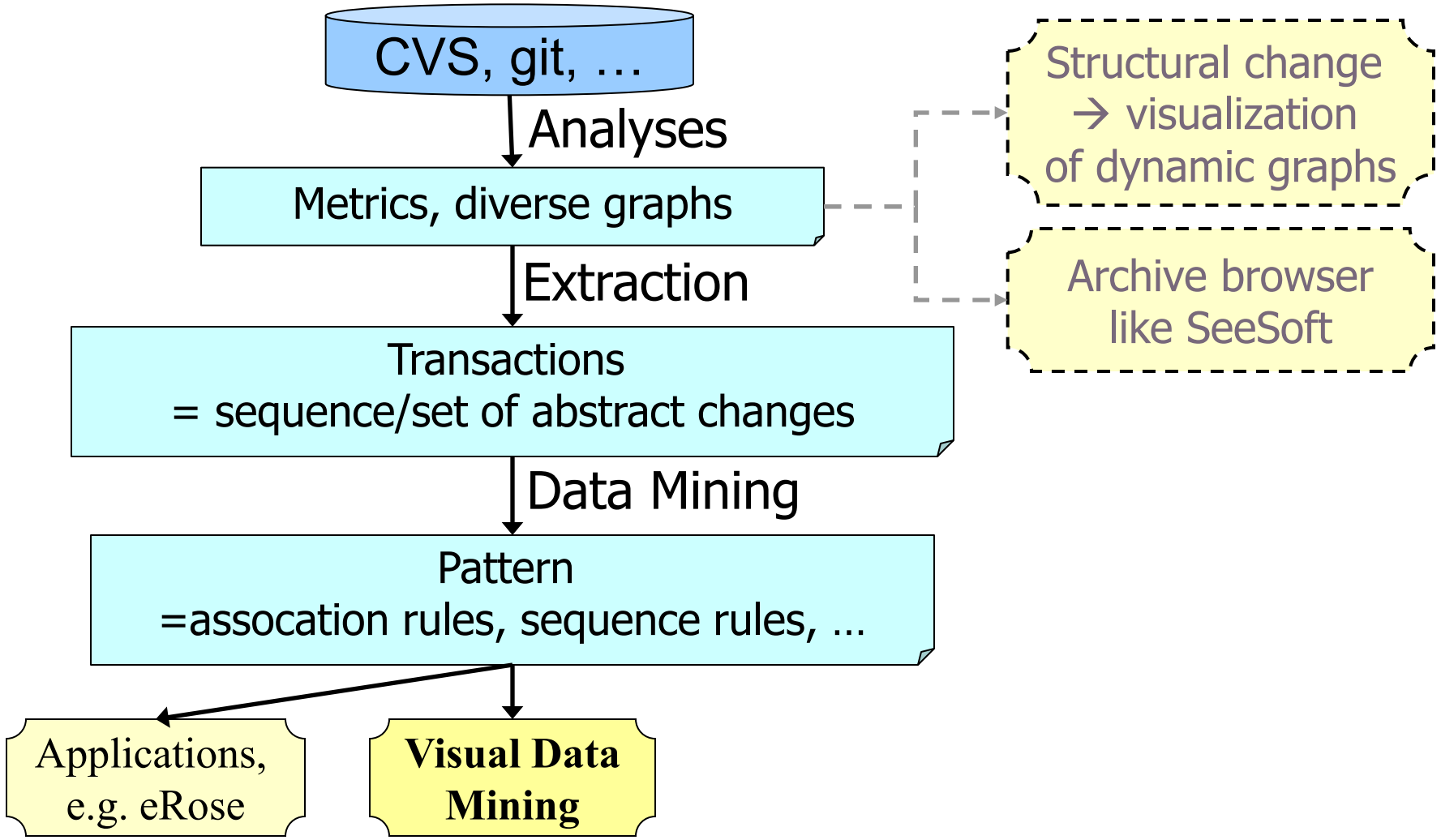


Finding Evolution Patterns

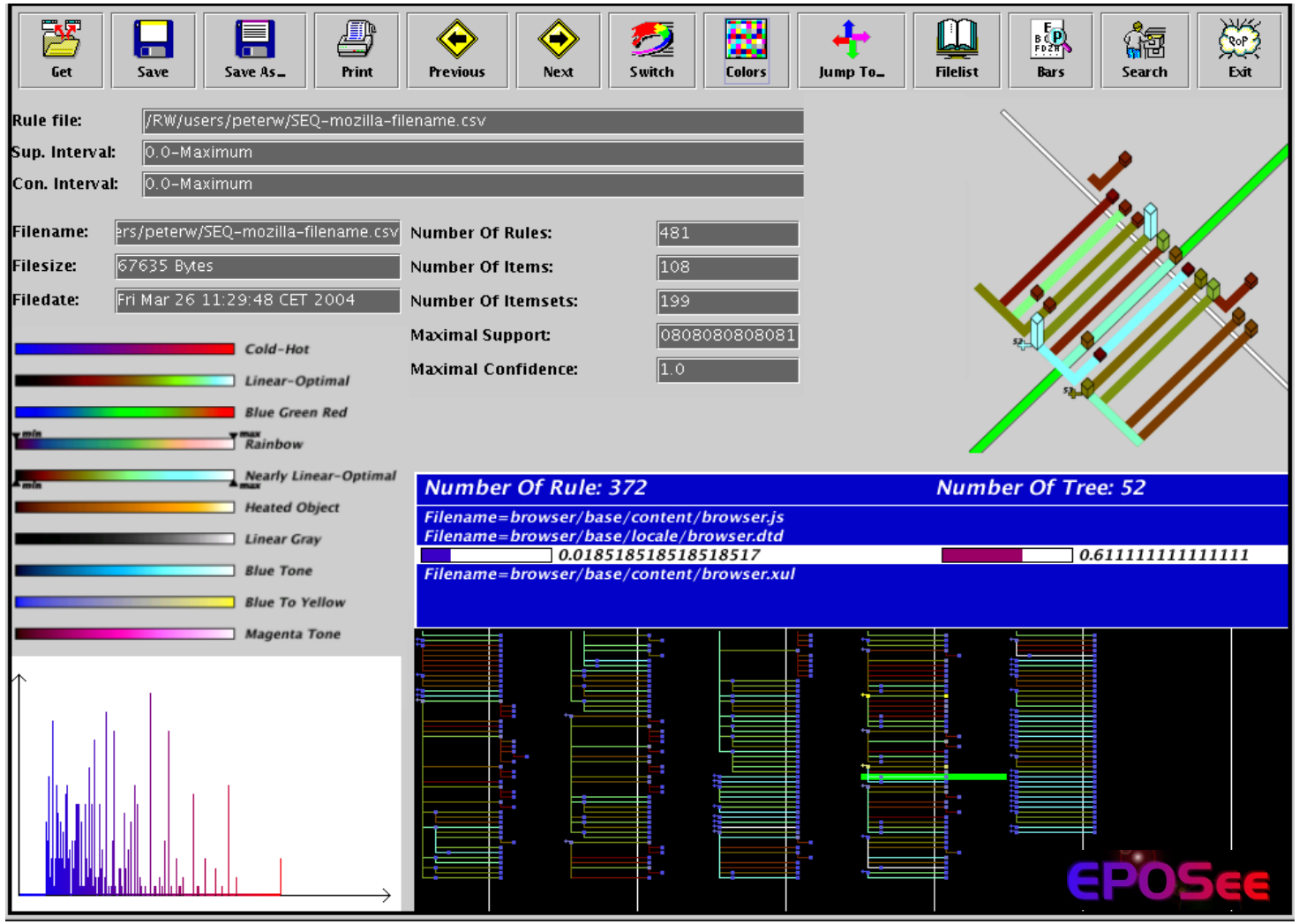
- Data Mining
  - There exist many techniques and types of rules/patterns.

- Visual Data Mining
  - Data Mining often yields a great many rules
  - Visualization is used to explore these rules
    - global observations (e.g. recurrent structures)
    - local observations (e.g. outliers)

Mining process



EPOSee



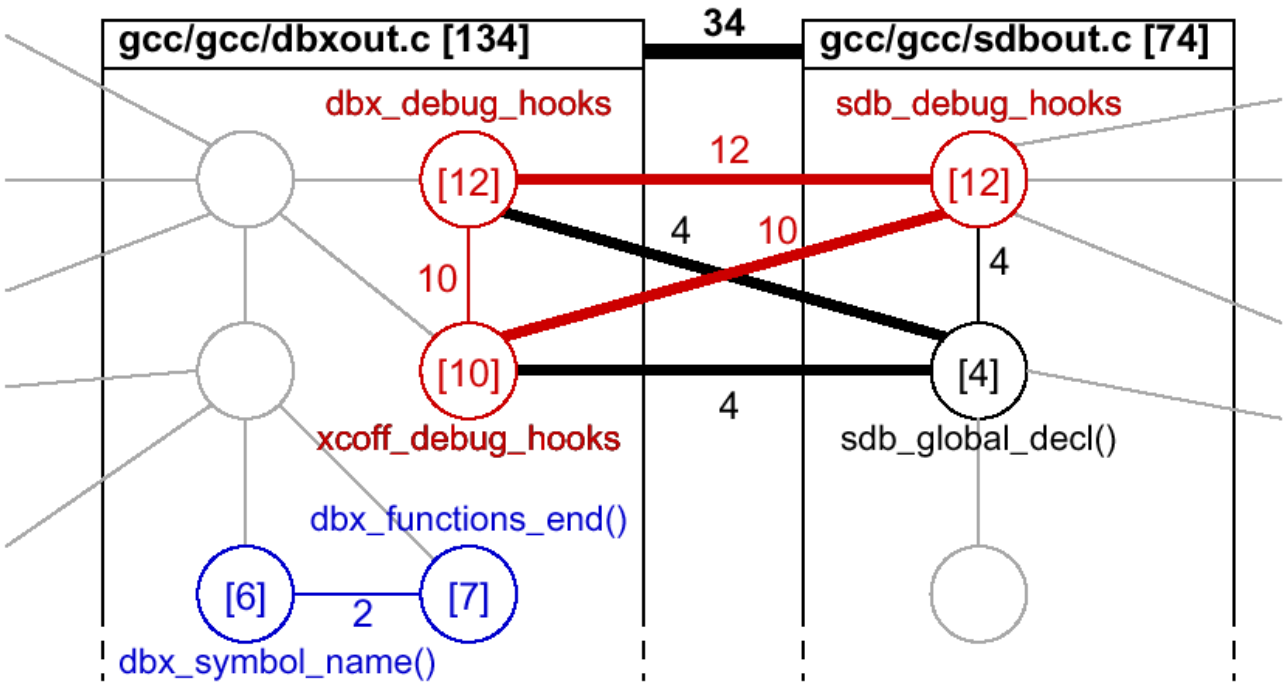
combines various interactive visualization techniques to explore association and sequence rules

Source: **Visual data mining in software archives.** M. Burch, S. Diehl, and P. Weißgerber. In *Proceedings of the ACM Symposium on Software visualization* (SoftVis '05), 2005.

Evolutionary Coupling

- Two software artifacts (file, function, class, etc.) are evolutionary coupled, if they have been changed together (at the same time/within the same time period).

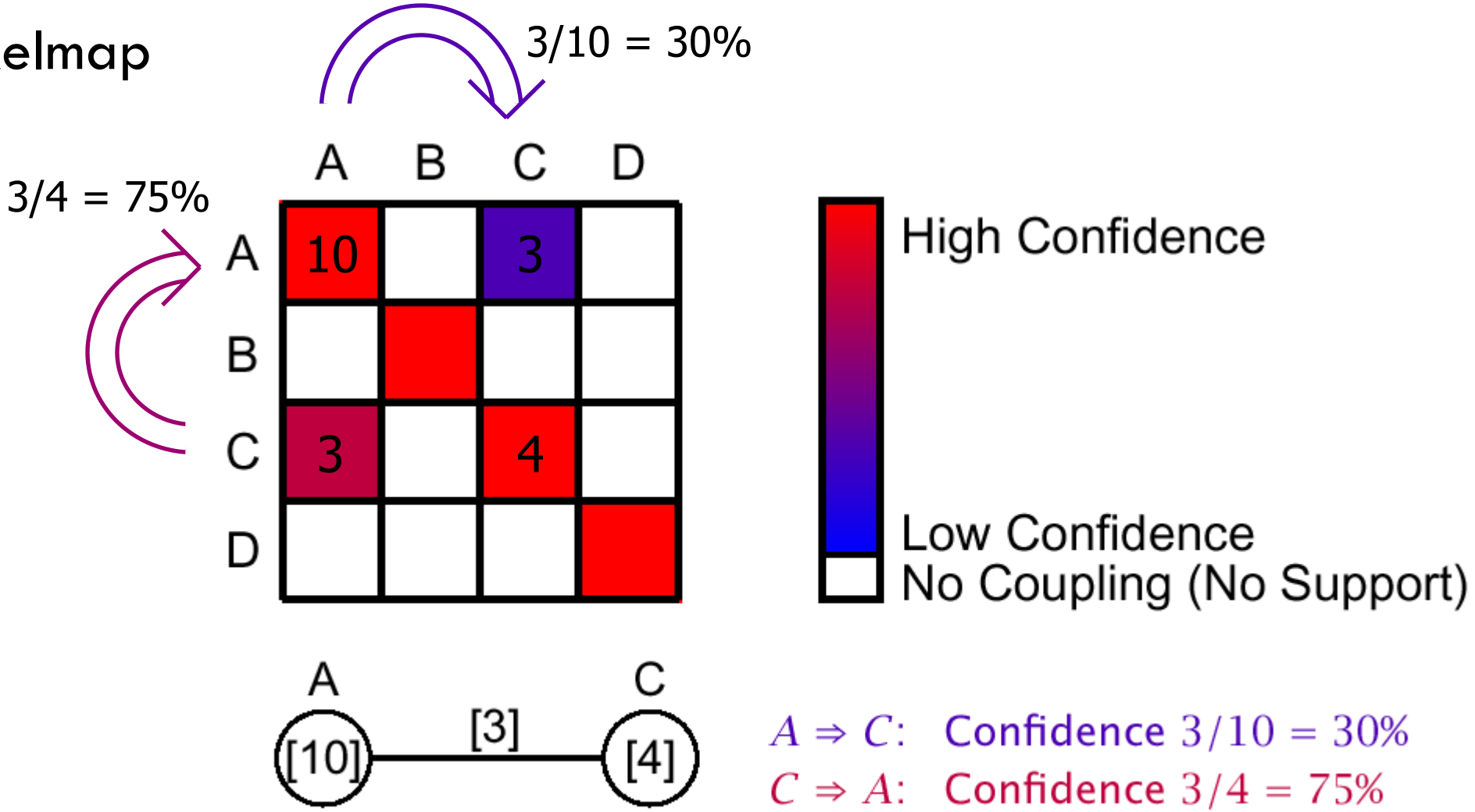
Evolutionary Coupling vs. Logical Coupling



Source: **How History Justifies System Architecture (or not).** T. Zimmermann, S. Diehl, A. Zeller, International Workshop on Principles of Software Evolution IWPSE 2003, Helsinki, Finland, September 2003

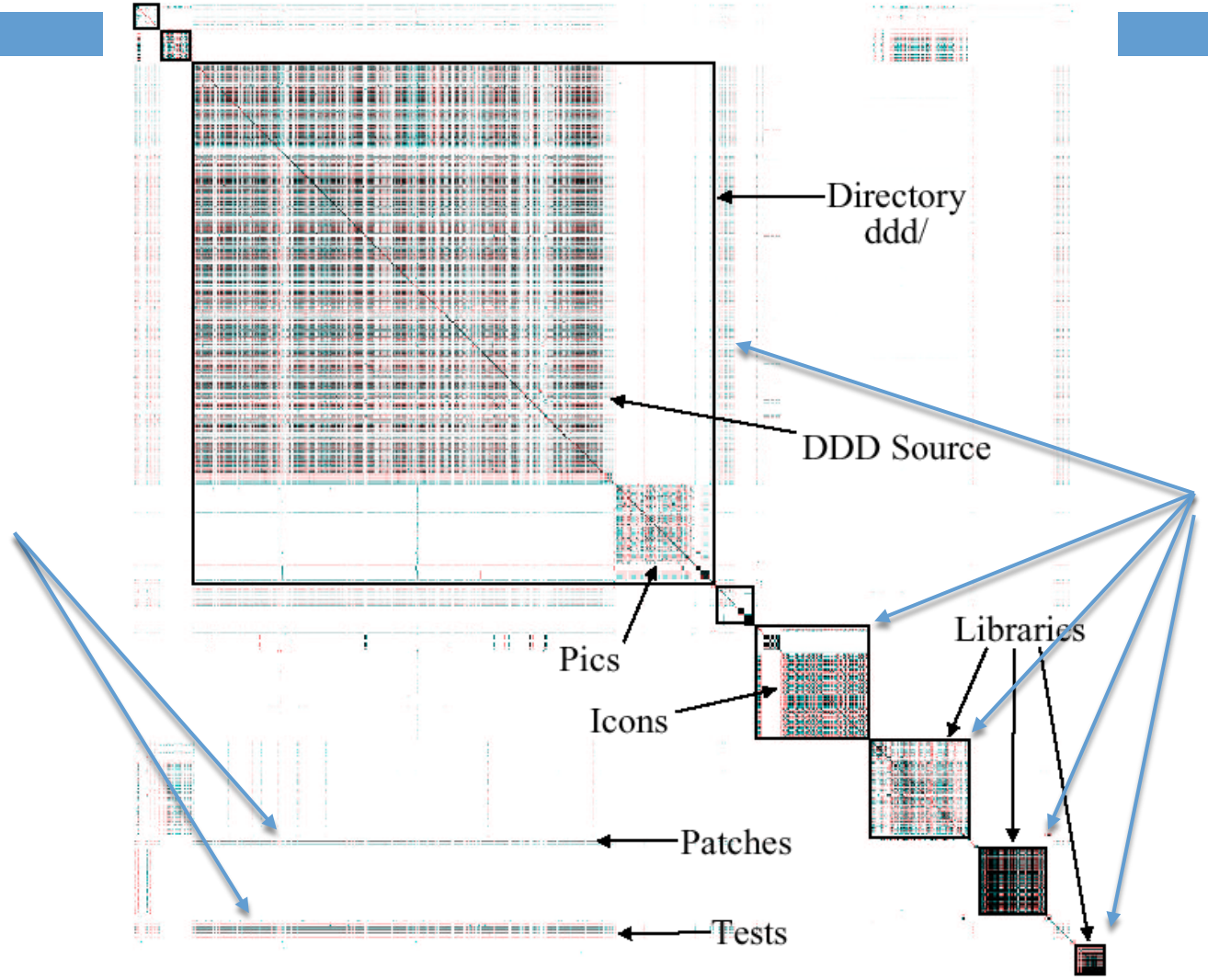
Visualization of evolutionary coupling

- Pixelmap



Pixelmap of DDD

EPOSee



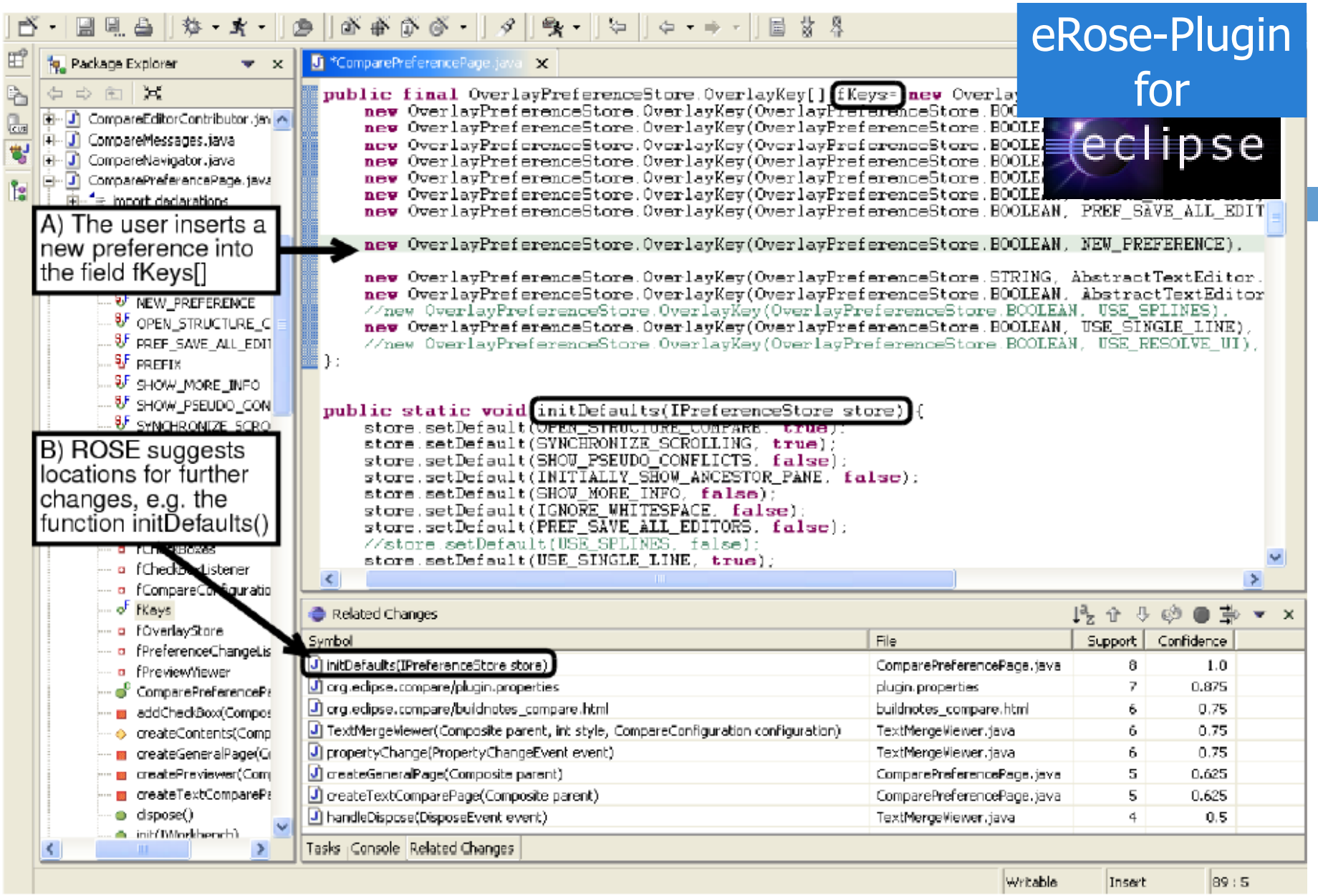
Outliers

- Possible conflict with module structure
- Has to be checked with other methods/tools

Clusters (along the diagonal)

Do evolutionary coupling and module structure correspond?

Insights gained with visual data mining motivated the design of a tool for programmers.



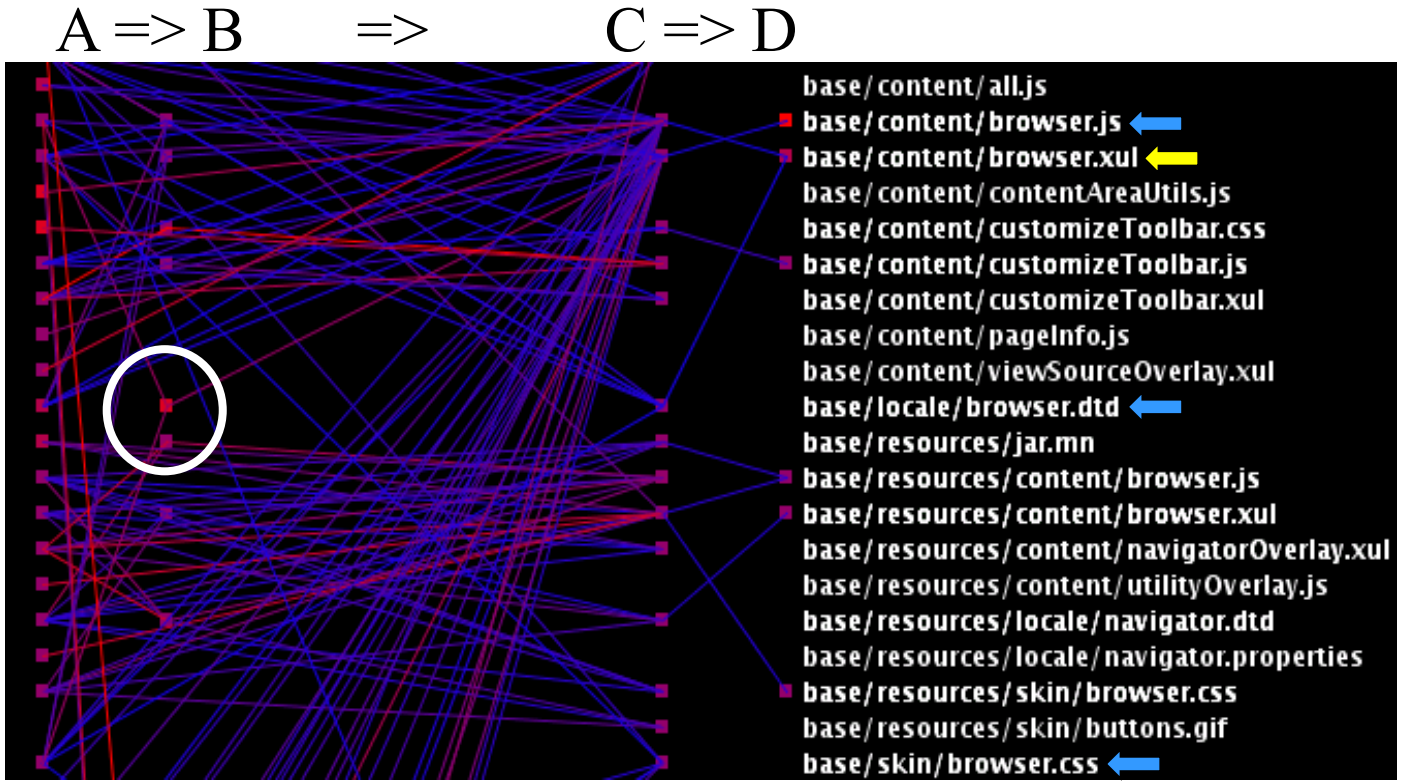
Source: **Mining Version Histories to Guide Software Changes.** T. Zimmermann, P. Weißgerber, S. Diehl and A. Zeller, in Proceedings of International Conference on Software Engineering (ICSE'04), Edinburgh, UK, 2004

Extra slides – not covered in lecture !

EPOSee

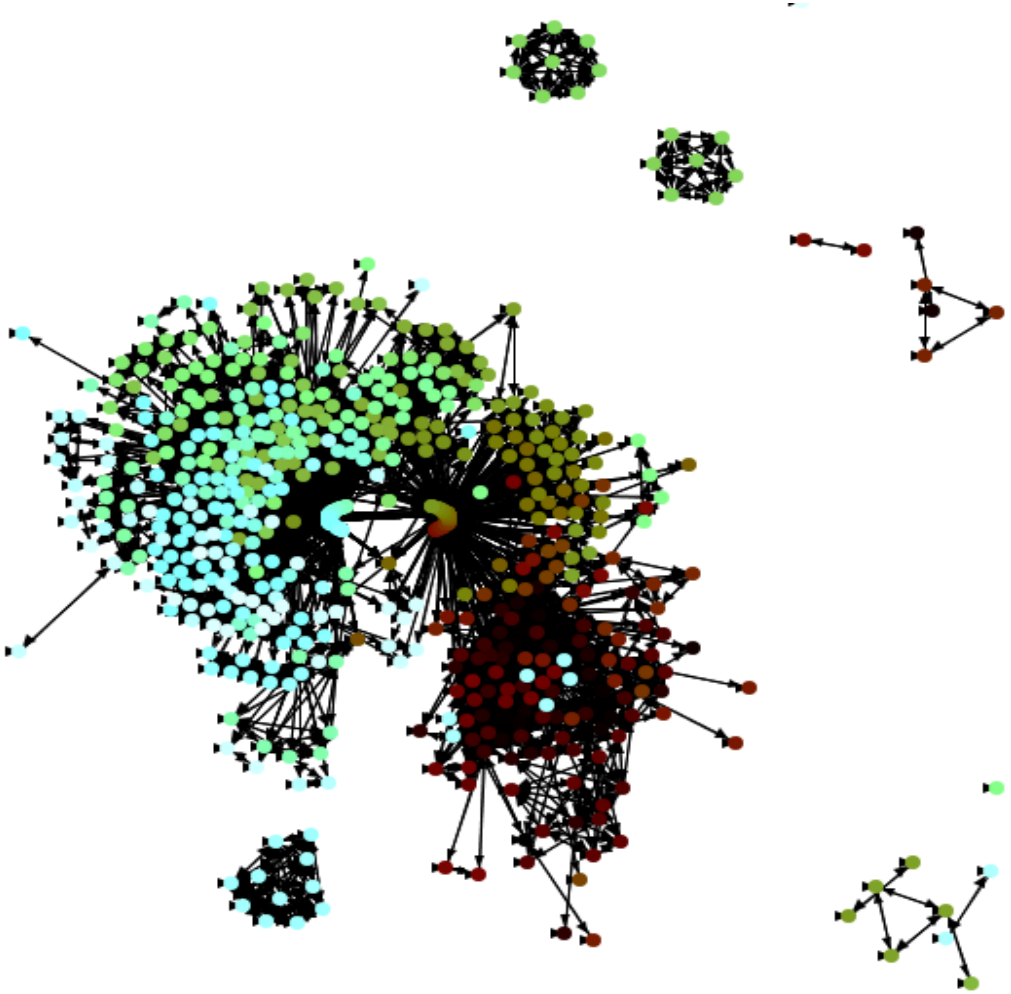
Sequence rules

- Assoziation rules
  - Only concurrent changes
- Sequence rules
  - Subsequent changes
  - Antecedence and consequence are sequences:
    - „First A, than B, …“
  - Consequence follows antecedence in time
  - Possible interpretation: causality



Base directory of Mozilla Firefox

Visualization of evolutionary coupling



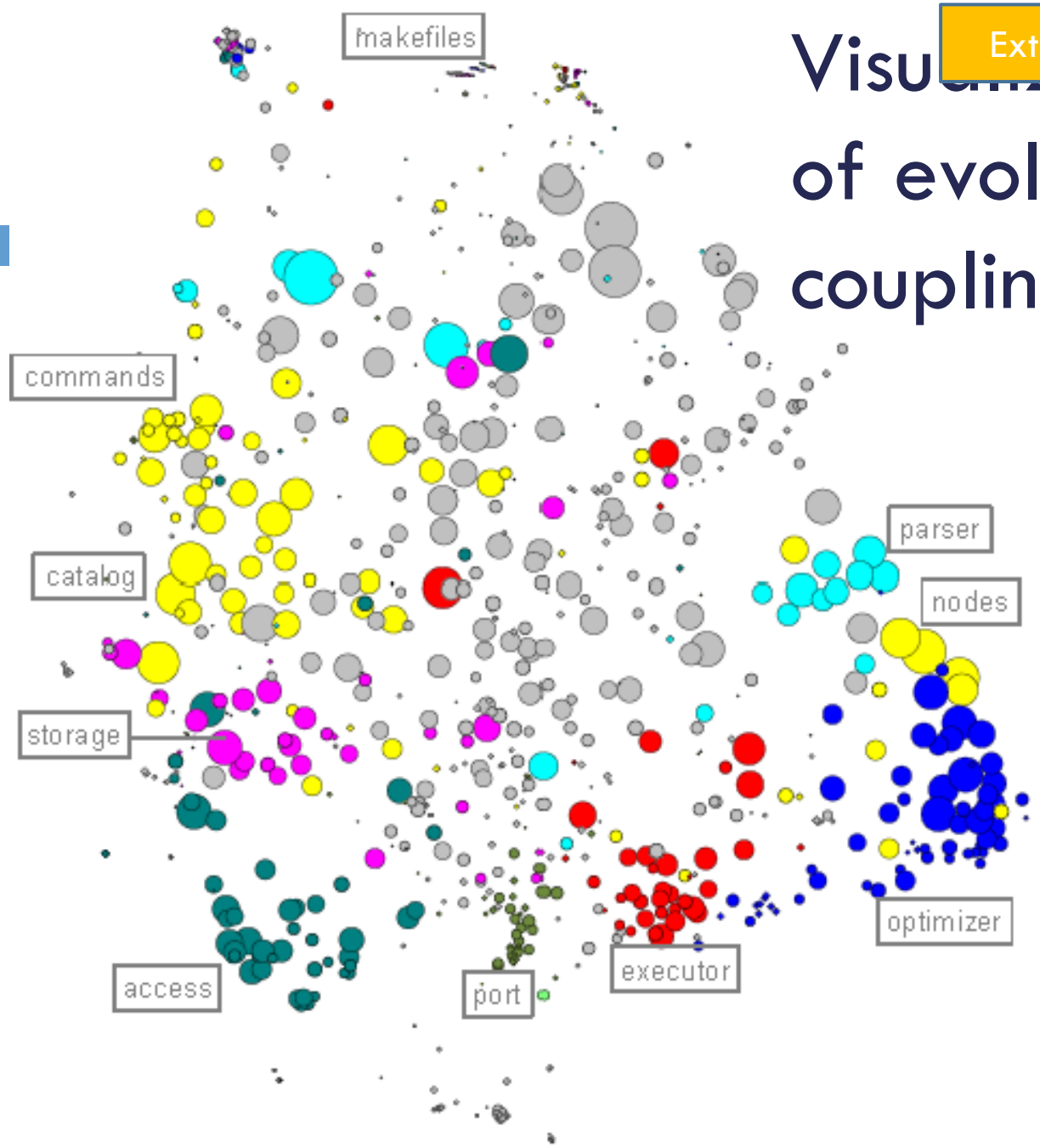
- Strongly coupled items are drawn close to each other (force-directed layout)
- Hierarchical Information used to color the nodes (files in the same directory have same color)

Support-Graph of Mozilla Firefox (weight = number of times files were changed together)

Extra slides – not covered in lecture !

Visualization of evolutionary coupling

CCVisu

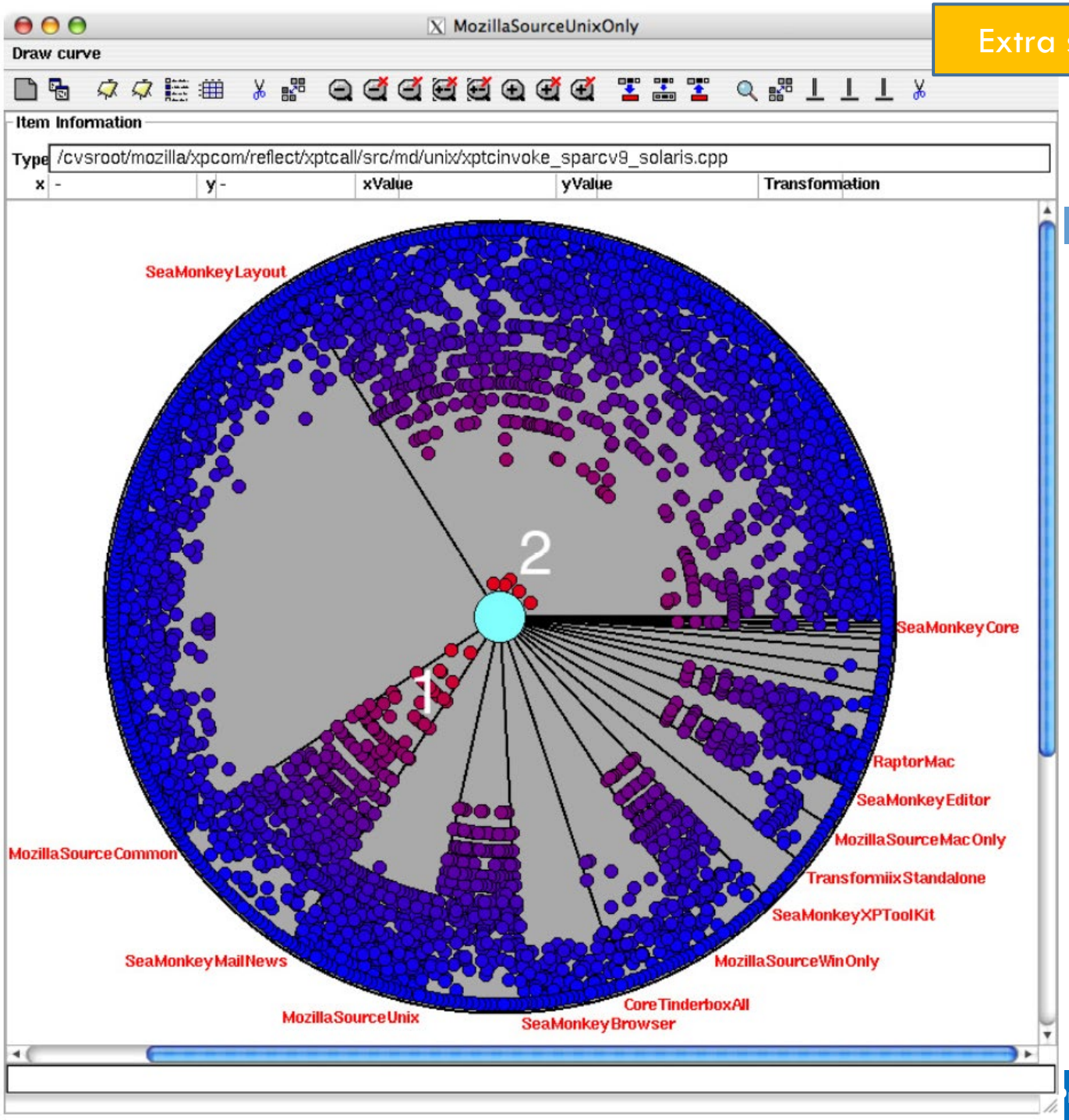


- Backend von PostgreSQL
- Energy-based clustering with CCVisu
- Edges are not drawn

- (Source: Dirk Beyer, MSR Challenge 06)

Extra slides – not covered in lecture !

Radar



Shows coupling with respect to a focus set (of files) in the center. The closer a file is to the center, the stronger it is coupled with the focus set.

Source: **The Evolution Radar: Visualizing Integrated Logical Coupling Information.** *Marco D'Ambros, Michele Lanza, Mircea Lungu.* In Proceedings of MSR 2006 (3rd International Workshop on Mining Software Repositories), 2006.

Summary

Visualizing the Evolution of Software

- Many techniques for visualizing the temporal change of structure, metrics and text (source code)
- Visualizations help users to explore and analyze the data

(Visual) Data Mining

- Tools, in particular recommender systems, analyse “historical” information from software archives.